

Entering the Urban Mobility Market: Demand Prediction and Smart Charging for an Electric Ride-Hailing Fleet

Advanced Analytics and Applications

Team 02



Authors:

Anthony Ge (7381058)

Hendrik Mehl (7410290)

Niklas Eichholz (7354459)

Yannick Herrmann (7409728)

Contact: age@smail.uni-koeln.de

Supervisor: Univ.-Prof. Dr. Wolfgang Ketter

Co-Supervisor: Janik Muires

Chair of Information Systems for Sustainable Society
Faculty of Management, Economics and Social Sciences
University of Cologne

2026-07-26

Table of contents

1	Problem Description	1
2	Data Description	1
2.1	Trip Data	2
2.2	Supplementary Data	2
2.3	Aggregated Demand Dataset	3
3	Data Analytics	3
3.1	Descriptive Analysis	3
3.2	Demand Prediction	4
3.2.1	Prediction using Support Vector Machines	5
3.2.2	Analysis using Neural Networks	6
3.2.3	Model Comparison: SVM vs. Neural Networks	8
3.3	Analysis using Reinforcement Learning	9
4	Conclusion	10
5	Appendix A: Figures	11
6	Appendix B: Use of LLMs	46

List of Figures

1	Raw Chicago Taxi Trip Data Column Descriptions	11
2	Weather Data Columns	12
3	Spatial resolution available per trip endpoint	12
4	Boxplot of taxi trip attributes to detect outliers	13
5	census tracts with trips outside of Chicago	14
6	Trip distribution plots at the community area level	15
7	Trip distribution plots for H3 hexagons at multiple levels	16
8	Trip distribution for H3 Hexagons: centroids vs area-weighted	17
9	POI features by community area: distance to nearest airport, restaurant density, hotel density, and total POI density.	18
10	Trip distribution by H3 hexagon	19
11	Trip distribution by community area	20
12	Spatial KDE of Taxi Pickup Demand	21
13	Spatial KDE of Taxi Pickup Demand by Time of Day	22
14	Average price by H3 hexagon	23
15	Average price by community area	24
16	Average trip miles by H3 hexagon	25
17	Average trip miles by community area	26
18	Trip Duration by H3 hexagon	27
19	Trip Duration by community area	28
20	Idle Time by H3 hexagon	29
21	Idle Time by community area	30
22	Tip Rate by H3 hexagon	31
23	Tip Rate by community area	32
24	Start Time by H3 hexagon	33
25	Start Time by community area	34
26	Chicago Ride-Hailing Demand by Hour of Day	34

27	Chicago Ride-Hailing Demand by Day of Week	35
28	Chicago Ride-Hailing Demand by Month	35
29	Chicago Ride-Hailing Demand Heatmap: Day of Week vs Hour of Day	36
30	Average trip count by Hour and Rain Condition	36
31	Average tip rate by Hour and Rain Condition	37
32	Average idle time by Hour and Rain Condition	37
33	Distribution of Taxi Idle Times before a Trip	38
34	Average fare per mile by Hour and Rain Condition	38
35	Average trip duration by Hour and Rain Condition	39
36	Average trip distance by Hour and Rain Condition	40
37	Grid Search Results	41
38	Model Results	42
39	Hurdle Results	42
40	Spatio-temporal model comparison (R^2)	43
41	Spatio-temporal model comparison (MAE)	44
42	Charging cost as a exponential function of charging power	44
43	Transition structure of the EV-charging MDP	45
44	Learning curves of Monte Carlo and Q-learning	45
45	The exact optimal policy next to policies learned by Monte Carlo and Q-learning	45

1 Problem Description

The smart mobility market is highly competitive, and current reports indicate that ride-hailing, specifically with autonomous and electric vehicles will dominate it in the near future. This development opens a substantial opportunity for traditional car manufacturers to move beyond vehicle sales and establish themselves as competitive mobility service providers.

Our client, a renowned German car manufacturer, seeks to enter the US ride-hailing market with a fully electrified fleet. The client, however, lacks operational expertise in managing electric vehicle fleets in an urban ride-hailing context. Several challenges follow directly from that gap: how to reduce energy costs, how to manage fleet charging schedules, and how to rebalance and relocate vehicles, all reactively in real time and proactively in advance.

Each of these questions reduces to the same underlying ability: knowing where and when how much demand will occur. Charging schedules can only be optimized against a known window of low utilization, rebalancing is only possible if the imbalance is anticipated in advance, and infrastructure investment depends on where demand concentrates over the long run. Without a reliable demand forecast, every decision becomes purely hypothetical.

To address this, we analyze historical taxi trip data from Chicago as a proxy for ride-hailing demand, in three stages. First, a descriptive analysis examines how taxi demand varies across spatial and temporal resolutions, establishing a broad picture of the relationships between variables in the data. Second, we build predictive models that forecast trip demand, for example, expected demand on a Sunday morning in the city outskirts, enabling the client to shape its operational strategy e.g. where to deploy additional vehicles. Third, we develop an intelligent charging strategy using reinforcement learning to minimize charging costs while ensuring sufficient vehicle range. Together, these analyses enable the client to make data-driven decisions on fleet deployment, driver scheduling, and charging infrastructure investment.

2 Data Description

To approach this task, data from multiple sources was used. The primary dataset used in this analysis is the Chicago Taxi Trips dataset, which is publicly available via the [Chicago Data Portal](#) and contains taxi trips from 2024 onwards. The Taxi Trip Dataset contains one trip per row, with timestamps rounded to the nearest 15 minutes. Since the full dataset starting from 2024 contains over 16 million rows, the dataset was limited to only include trips from 2025, resulting in a raw dataset of roughly 6.8 million rows. Important attributes include trip and taxi identifiers, the timestamp of the trip, centroid coordinates of the community area or census tract where the trip took place and multiple attributes about the trips properties like price or length (Figure 1).

Furthermore, hourly Chicago weather data was obtained from the [Open-Meteo API](#) for the period of January 2024 to January 2026. The initial dataset includes roughly 17.500 entries and key variables of the dataset include temperature, precipitation and sunshine duration (Figure 2).

Points-of-Interest data for Chicago was gathered from OpenStreetMap via the [Overpass API](#), covering nine categories: airports, train stations, stadiums, restaurants, bars and clubs, hotels, hospitals, universities, and tourist attractions. These raw POI locations are subsequently aggregated into distance and density features for multiple spatial resolutions during data preparation.

In order to be able to analyse the data on census tract and community area level, additional geometry data containing the shapes of these areas was obtained. These datasets contain the geometry for Chicago's 77 community areas and 878 census tracts.

2.1 Trip Data

The dataset was first inspected to get an overview of the datatypes and find potential issues. This revealed that several numeric columns were incorrectly stored as strings and that the timestamp columns were not yet in datetime format. Both were corrected during preprocessing. Additionally, the dataset contained no duplicated rows or trip identifiers. Furthermore, several implausible trips were identified and removed during cleaning. These included trips with zero or negative distance (~528.000 trips) and trips with a duration of less than 60 seconds (~381.000 trips), as these trips are likely erroneous. Overlapping trips with the same taxi identifier were also removed (~28.000 trips) based on the assumption that these result from recording errors. Price attributes were also validated by checking for and removing trips with zero or negative fares (1.455 trips) and whether there are negative tips, tolls or extras (0 trips).

The dataset was also checked for missing values, revealing that 0.18% of the non-spatial data was missing. Due to the negligible share, these rows were dropped. Because of privacy reasons, the spatial data showed many more missing values with census tract data missing for approximately 55% of trips. In these cases, the less granular community area is recorded instead, meaning spatial information is almost always available at either the census tract or community area level (Figure 3).

The numeric columns were checked for outliers using boxplots (Figure 4) and removed using the z-score as a statistical solution. Observations with a z-score exceeding a threshold of 2 in any numeric column were considered outliers and removed. A threshold of 2 was chosen over the more common value of 3 to remove a larger share of extreme values, which was deemed acceptable given the large remaining dataset size. This resulted in ~444.000 rows, or 7.18% of the data at that point being removed, leaving the set with roughly 5.7 million rows.

Starting from the outlier-cleaned trip data, spatial fields were finalised through a series of steps. First, a trip was removed if it lacked all spatial information (census tract, community area, coordinates) for its pickup or dropoff location. Second, trips starting or ending outside of Chicago (Figure 5) were dropped using the official community area boundaries as the city outline. Third, for all trips with missing community area information, this information was filled using the census tract data, such that the community area is available for all trips and can be used as spatial unit (Figure 6). Additionally, H3 hexagons were visualized at multiple levels, 5,6,7 and 8 (Figure 7). Resolution 5 was too coarse (only 8 hexagons for Chicago) and resolution 8 implied more spatial precision than the data supports given that coordinates are community area or census tract centroids. Based on that, hexagon IDs at resolution 6 and 7 were assigned to each row with a special approach. Since the trips only carry the single centroid coordinate of their community area, assigning hexagons directly from these coordinates would make all of these trips collapse onto the same point, potentially leaving hexagons with zero trips and an uneven distribution. To avoid this, trips were instead spread out across the whole community area before being assigned to a hexagon, resulting in a smoother, way more realistic distribution across the map (Figure 8). Finally, together with the weather dataset, a merged dataset based on the starting hour of the trip was created.

2.2 Supplementary Data

Before the merge, the weather dataset was filtered to the year 2025 to match the trip data, and timestamps were converted from UTC to Chicago local time. A duplicate timestamp and a missing timestamp were identified, which can both be attributed to the daylight saving time transition, and were therefore retained as-is. No further cleaning was required. For the POI data, the raw point locations were aggregated into spatial features for each community area and H3 hexagon at resolutions 6 and 7. Depending on the category, either a distance or a density feature

was computed. For categories where proximity is most relevant, the distance to the nearest POI was calculated, while for categories where concentration matters more, the number of POIs per km² within each spatial unit was used (Figure 9).

2.3 Aggregated Demand Dataset

For predictive modeling, the merged dataset was aggregated into a demand dataset where each row represents one spatio-temporal unit. Trips were grouped by time bucket and spatial cell. To ensure complete coverage, zero-demand rows were added for every spatial cell and time bucket combination with no recorded trips. Four temporal resolutions (1h, 2h, 6h, 24h) and three spatial resolutions (H3 resolution 6, H3 resolution 7, community area) were combined, resulting in 12 datasets. Each row was enriched with temporal features (hour of day, day of week, month, season, public holidays), city-wide weather conditions averaged per time bucket, and static POI features describing the character of each spatial cell. Additionally, idle time was computed as the time between consecutive trips for the same taxi. Gaps exceeding 4 hours were treated as off-duty periods and excluded. Cyclical temporal features such as hour of day, day of week, and month were encoded using sine and cosine transformations to preserve their circular nature.

3 Data Analytics

3.1 Descriptive Analysis

Visualizing trips at different spatial resolutions (community areas and H3 hexagons 6 and 7) shows how trip demand is heavily concentrated in the downtown and in the northwest at the O'Hare Airport (Figure 10, Figure 11). This makes sense due to the high concentration of offices, hotels, restaurants and other points of interest in the downtown area and taking into account that the O'Hare Airport is one of the busiest in the world. This same pattern can also be seen when analyzing demand through a kernel density estimation (Figure 12). Since the dataset only provides each trip's community-area centroid, we improved the KDE by scattering each area's trips uniformly at random within its polygon before smoothing. Visualizing the KDE at different times during the day does not show any significant changes to the pattern (Figure 13). The contrast between these high-demand zones and the rest of the city is visible in all spatial resolutions. There is no noticeable difference when comparing demand patterns between pickup and dropoff coordinates, which is expected, since every pickup must correspond to a dropoff, so demand is symmetric across origins and destinations.

Additionally, multiple trip attributes were spatially visualized at the same spatial resolutions. Visualizing the average fare (Figure 14, Figure 15) and trip miles (Figure 16, Figure 17) per trip shows a very similar pattern, which is expected, since trip distance directly affects the price. They are both highest for pickups at O'Hare airport, which makes intuitive sense, since passengers flying into Chicago are typically traveling to hotels, offices, or residences spread across the city, resulting in long trips. The Midway airport in the southwest shows a similar but less pronounced effect. The downtown core shows consistently short pickup distances and lower prices, reflecting short urban trips between nearby destinations within the city center. This contrast between airport and downtown is more pronounced in the pickup maps than the dropoff maps. A very similar pattern can be observed when visualizing trip duration (Figure 18, Figure 19). Displaying the average idle time shows highest values around the two airports (Figure 20, Figure 21). Airport pickups are regulated through designated taxi holding lots where drivers must queue and wait for dispatch, artificially inflating idle time compared to city street hailing where drivers can cruise actively. The dropoff maps show no comparable hotspots. Drivers dropping off at the airport immediately join the holding queue, but passengers picked up at the

airport are distributed across the entire city, so no single dropoff zone accumulates unusual idle time. Additionally, the tip rate (tips/fare) was spatially analyzed (Figure 22, Figure 23). The tip rate is highest for pickups at O'Hare, aligning with the trip distance finding. Airport passengers taking long trips tend to tip more generously, possibly because the fare is higher and tipping feels more expected for longer journeys. Midway airport shows a similar effect. Additionally, downtown shows an increased tip-rate. This reflects the demographic of passengers in that area, which include tourists, business travelers, and higher-income residents who predominantly tip more generously.

Spatially comparing the trip start times between pickup and dropoff also shows an interesting pattern (Figure 24, Figure 25). It shows how the airports stand out as having very late average starting hours compared to the rest of the city (ca. 17:00pm), while, looking at trip dropoffs, the airports stand out as having the earliest start hours. Additionally, when examining the dropoff maps, it stands out that there is a high concentration of late trips in the northern part of Chicago, which may be attributed to a higher concentration of destinations like restaurants.

Beyond spatial patterns, the trip data was also analysed across temporal dimensions. Taxi demand follows a clear daily pattern with a morning peak of around 8-9h and an evening peak at 17h (Figure 26). This pattern reflects the start and end of the work day. At the weekly level (Figure 27), weekdays are shown to have the highest demand with Wednesday and Thursday being the busiest days, showing that Chicago's taxi demand is primarily business commuter driven. Monthly (Figure 28) demand shows how demand is lowest in winter, with peaks in May, June and October. Combining the hourly and daily views in a heatmap (Figure 29) reveals that weekend demand lacks the morning and evening peaks seen on weekdays, instead being spread more evenly across the day.

Weather conditions also influence several trip attributes. Rain consistently increases trip demand across all hours of the day (Figure 30), as people who might otherwise walk or use bicycles switch to taxis in bad weather. Rain also slightly increases the tip rate (Figure 31) and reduces the idle time (Figure 32). Here, the general pattern is also interesting with the tip rate being highest in the morning and idle time being at its lowest when the workday starts and ends. Visualizing the idle time distribution (Figure 33) additionally reveals how most drivers find new passengers relatively quickly ($< 20\text{min}$). Additionally, analyzing fare per mile also shows the highest values during the morning and evening, which may reflect more stop-and-go traffic increasing time-based fare components relative to distance (Figure 34). Trip distance and duration both show an interesting spike in the early morning (Figure 35, Figure 36), which can be attributed to airport runs. Rain has a very small effect here, slightly increasing the duration and decreasing the distance.

3.2 Demand Prediction

This chapter turns to the underlying question our business client asks: are machine learning models worth their keep in predicting ride-hailing demand. We approach it the way a fleet operator would, weighing predictive gains against the practical cost of adopting more complex methods.

Two questions organize the analysis. First, can support vector machines and neural networks predict demand reliably, and do they improve compared to simple historical averages by enough to justify the effort of building, tuning, and maintaining them?

Second, how should the data be represented? We compare uniform hexagonal grid cells against administrative community areas, and hourly demand slices against coarser time buckets, to determine which spatial and temporal resolution best supports accurate prediction.

To assess the following models in a comparable way, we carry two measures through the chapter. MAE (mean absolute error) captures the average miss in trips for a given area and time, while R^2 measures how much of the genuine variation in demand a model reproduces (1.0 marking a perfect fit). Additionally, we employ a benchmark using historical averages: for each spatial cell, the mean demand of all training observations sharing its hour of day and day of week, computed on the training split only and applied unchanged to the test period, so that it captures the recurring weekly pattern without access to future data.

3.2.1 Prediction using Support Vector Machines

We modeled Chicago taxi demand using Support Vector Machines (SVM) at an H3 hexagon (resolution 6) 1-hour spatio-temporal resolution to optimize high-frequency fleet dispatch logistics.

To begin with, we had to select suitable features and preprocess the data. To prevent critical data leakage and establish strict structural boundaries, we excluded nine variables that are only known after a trip concludes, such as the number of active vehicles, the average idle times of vehicles, the average trip duration. Next, to avoid temporal leakage, the dataset of ~289,000 entries was partitioned using a strict chronological split (80% train / 20% test), reserving the final ~58,000 intervals for validation. Finally, the location identifiers (community areas and H3 hexagons) are one-hot encoded, allowing the model to learn a separate intercept for each neighborhood.

To establish an initial understanding of feature importances, we conducted a linear coefficients analysis. This revealed that Point of Interest (POI) data strongly drives trip demand, led by density metrics such as number of hotels per squared km and number of attractions per km². Conversely, local weather metrics like the apparent temperature and amount of rain represent low-impact statistical noise.

As an initial benchmark, we evaluated a Linear Support Vector Regression model without a kernel. This baseline underperformed, yielding an R^2 of 0.6134 (MAE = 13.05 trips). Its primary limitation was a severe boundary infraction: 34.52% of predictions fell below zero, exposing the linear plane’s inability to respect non-negative demand thresholds.

Subsequent grid-search optimization (detailed in Figure 37) clearly identified the Radial Basis Function (RBF) kernel as the most promising approach. It also revealed 1000 as most promising value for the hyperparameter C. However, exact RBF kernel training scales cubically ($O(N^3)$) with dataset size, rendering standard training computationally intractable on our hardware. To bypass this bottleneck, we deployed a Nystro m low-rank approximation to project the infinite-dimensional RBF space onto a compact set of landmark points. We evaluated different landmark counts alongside models with and without weather features, given that our previous linear coefficient analysis flagged weather metrics as marginal. As shown in Figure 38, our top-performing configuration excluded weather data entirely, climbing to 0.8734 R^2 and driving the MAE down to 6.49 trips.

While these performance metrics represent a robust fit, especially given a highly efficient training runtime of approximately 7.5 minutes, the model still produced negative predictions for 22.28% of intervals, which required clipping to zero. To naturally enforce physical boundary constraints, we developed a two-Stage Hurdle SVM Framework. Stage 1 (The Gatekeeper) utilizes an RBF-approximated LinearSVC binary classifier to map the active-demand perimeter (1 if active, 0 if dead). Stage 2 (The Volume Engine) then runs an RBF-approximated LinearSVR trained exclusively on active instances, shielding the regression plane from zero-floor distortions (see Figure 39 for the exact results).

Despite a minor structural tendency to generate false zeros during low-demand hours, this optimized No Weather Hurdle Model emerged as our champion architecture, achieving a 0.8751 R^2 and a 6.44 trips MAE. Crucially, it maximizes variance capture while mathematically guaranteeing 0% negative boundary violations, rendering it highly viable for production deployment.

We also explored how performance scales across different spatio-temporal resolutions. Decreasing temporal resolution (aggregating from 1-hour to 6-hour and 24-hour windows) smooths high-frequency demand surges, naturally boosting variance capture. The 6-Hour Hexagon (No Weather) model secured our highest raw fit at 0.9042 R^2 (MAE = 32.66 trips). However, absolute errors scale with the larger temporal footprint, which limits its utility for real-time dispatching.

Shifting the spatial unit from uniform geometric hexagons to irregular, administrative boundaries (Census Tracts/Community Areas) degrades true operational accuracy. The 24-Hour community area model yields a deceptively high 0.8929 R^2 but suffers a massive absolute error distortion of 57.12 trips MAE (compared to the 24-Hour Hexagon’s 26.98 MAE). This surge in MAE is a textbook demonstration of the Modifiable Areal Unit Problem (MAUP). It is driven primarily by the Scale Effect: because administrative Community Areas are larger on average ($\sim 7.68 \text{ km}^2$) than Resolution 7 hexagons ($\sim 5.16 \text{ km}^2$), they aggregate a much higher absolute volume of hourly trips, naturally inflating scale-dependent absolute metrics like MAE. Furthermore, the Zone Effect introduces irregular boundaries and mismatched spatial centroids, warping the distance metrics calculated on the SVM hyperplane. A full comparison of models can be found in Figure 40 and Figure 41.

3.2.2 Analysis using Neural Networks

This chapter analyzes the same demand prediction problem using feed-forward neural networks. We compared three network designs that differ in size: a baseline (2 layers of 64 units), a deeper model (8 layers of 64) and a wider model (2 layers of 256), each evaluated across four time buckets (1h, 2h, 6h, 24h) and both spatial units (community areas vs. H3 hexagons). More layers or wider layers mean more capacity to represent complicated patterns, but also more opportunity to latch onto noise. We tuned each architecture’s hyperparameters via grid search, keeping the configuration that delivered the best results for the least added complexity. Unlike the SVMs, neural networks let us handle location differently: instead of one-hot encoding, we can train a dedicated embedding layer that maps each hexagon or community area to a learned vector. This lets the network discover its own representation of spatial similarity, placing neighborhoods with comparable demand patterns close together in the embedding space rather than treating every location as an isolated binary feature.

A very consequential choice was what the model is asked to optimize. Taxi demand is count data: it is never negative, it is often zero, and its swings grow larger as demand grows. A standard error measure would ignore all three properties and happily predict negative trips. We therefore trained the 1h, 2h and 6h models on a negative binomial objective, which is built for exactly this kind of count data and guarantees positive predictions. There is a catch: the network produces its answer on a compressed scale and then expands it back to actual trip counts, and because that expansion is exponential, a small error on the compressed scale becomes a large error in trips. At daily aggregation, where demand averages roughly 180 trips per community area and 420 per hexagon, a modest overshoot can turn into an absurd forecast and destabilize training. The 24 h models were therefore trained on a more robust objective instead. As a result, their internal training values are not directly comparable to the shorter horizons; only the trip-count metrics reported below can be compared across all time buckets.

The table below shows the best network for each combination of spatial and temporal resolution, against its direct competitor, the historical average. Bold marks the winner on each

metric.

Resolution	Model	Loss	MAE	RMSE	R^2	MAPE
CA — 1h	deep_tuned	NB2	3.16	13.20	0.836	61.1
CA — 1h	<i>Historical mean</i>	—	3.48	13.92	0.817	70.4
CA — 2h	baseline_tuned	NB2	5.59	24.77	0.853	60.2
CA — 2h	<i>Historical mean</i>	—	6.54	26.95	0.825	78.7
CA — 6h	baseline_tuned	NB2	16.35	74.06	0.840	67.0
CA — 6h	<i>Historical mean</i>	—	17.87	72.68	0.846	85.2
CA — 24h	baseline_tuned	MAE	55.03	222.99	0.872	43.8
CA — 24h	<i>Historical mean</i>	—	53.67	203.11	0.894	69.5
Hex-6 — 1h	deep_tuned	NB2	6.23	20.32	0.869	66.3
Hex-6 — 1h	<i>Historical mean</i>	—	7.24	22.94	0.833	84.6
Hex-6 — 2h	deep_tuned	NB2	12.10	41.64	0.860	57.9
Hex-6 — 2h	<i>Historical mean</i>	—	13.88	44.52	0.840	86.0
Hex-6 — 6h	deep_tuned	NB2	36.40	128.24	0.839	57.8
Hex-6 — 6h	<i>Historical mean</i>	—	38.81	120.15	0.858	80.8
Hex-6 — 24h	deep_tuned	MAE	110.91	358.36	0.888	39.1
Hex-6 — 24h	<i>Historical mean</i>	—	115.95	329.77	0.905	64.2

Three findings stand out: The advantage shrinks as the time window widens. At 1h and 2h the tuned networks beat the historical average on every metric. At 6h they still win on average error (MAE and MAPE) but lose on RMSE and R^2 . The lookup edges ahead at 0.846 vs 0.840 for community areas and 0.858 vs 0.839 for hexagons. At 24 h the network’s advantage narrows to MAPE alone, and at CA-24h the benchmark wins outright, including on MAE.

The right model size depends on the spatial unit as well as the time bucket. The deeper network is strongest across all hexagon resolutions and at CA-1h, while at the coarser community-area buckets (2 h, 6 h, 24 h) the simpler baseline takes over. Left untuned, the deeper and wider models are the most unreliable of the three.

Weather features cost more than they contribute. Removing the weather block improved the model performance, which matches what the SVM feature analysis found independently. The reason is structural. The reason lies in the approach of aggregating the individual trips before they reach the network. After aggregation, the data follows a strongly repeating pattern: the same daily, weekly, monthly patterns holding remarkably steady across the year. Weather does the opposite: temperature, rain and wind swing enormously between January and July. So the model is handed inputs that vary heavily against a target that barely responds to them. Whatever genuine effect weather has on demand is small enough that the model cannot pin it down reliably: it ends up fitting incidental coincidences between weather and demand in the training period rather than a stable relationship, and those fitted patterns do not carry over to new data. Dropping the block removes that liability and leaves a smaller, cleaner input space, which is why the leaner models hold up just as well. It is a useful reminder that in a prediction task this regular, more data is not automatically more signal.

The hourly hexagon setup illustrates the whole pipeline. It covers 33 hexagons across roughly 289,000 area-hours; ~36% of the test rows have zero demand, and average demand is ~18 trips per hexagon-hour. The historical average sets the benchmark at $R^2 = 0.833$ with an MAE of 7.24 trips. A tuned basic Neural Network reaches $R^2 = 0.844$, above the benchmark. The tuned deep model reaches $R^2 = 0.869$ and MAE = 6.23 trips, roughly 14 % below the benchmark’s average error, and it earns that gain precisely where a lookup table is blind: on day-to-day deviations from the typical pattern for that hexagon and hour.

3.2.3 Model Comparison: SVM vs. Neural Networks

We evaluated whether a Feed-Forward Neural Network (NN) provides sufficient operational value over our champion Two-Stage Hurdle Support Vector Machine (SVM) for real-time fleet dispatching at our target resolution: 1-hour buckets across Resolution 6 H3 Hexagons (289,080 cell-hours; test mean demand ≈ 17.6 trips/hour).

Architecture / Model	R^2 Score	MAE (Trips)	RMSE (Trips)	Operational Complexity
Two-Stage Hurdle SVM (No Weather)	0.8751	6.44	20.31	Low–Moderate (~12 min train)
Deep Tuned NN (NB2 Loss)	0.8690	6.23	20.32	High (Multi-stage sweep)
Historical Mean Benchmark	0.8330	7.24	22.94	Zero (Lookup table)

At our core dispatch horizon, both advanced models comfortably outperform simple linear and historical benchmarks. The Deep Tuned NN achieves the lowest mean absolute error (6.23 trips/hour), marginally edging out the Hurdle SVM (6.44 trips/hour) by 0.21 trips per cell-hour. However, the Hurdle SVM outperforms the NN in global variance capture ($R^2 = 0.8751$ vs. 0.8690), while the two are effectively tied on peak error mitigation (RMSE = 20.31 vs. 20.32). The Hurdle SVR fits a continuous regression plane directly on active demand, tracking peak surges without numerical instability. Both models successfully eliminate impossible negative demand (0.0% infractions).

Is the Neural Network worth deploying? For a Chicago fleet operator managing real-time vehicle rebalancing, deploying a Neural Network over the Hurdle SVM is not justified:

1. Negligible Operational Edge: A marginal reduction of 0.21 trips/hour provides no practical advantage when rebalancing fleet vehicles in discrete units.
2. High Infrastructure & Training Overhead: Unlike the Hurdle SVM, which trains deterministically in ~12 minutes via Nystro m landmark approximation, NNs exhibit severe hyperparameter fragility (untuned variants collapsed to negative R^2) and require continuous re-tuning, GPU serving infrastructure, and monitoring against exponential prediction drift.
3. Inferior Surge Forecasting: Lower R^2 makes the NN less reliable for predicting critical high-density demand surges (e.g., event exits, airport waves) where fleet misallocations incur steep opportunity costs.

Our final deployment recommendation is to deploy the Two-Stage Hurdle SVM (No Weather). It maximizes variance capture ($R^2 = 0.8751$), natively prevents negative outputs, and ensures computational stability. We might suggest to retain the Historical Mean Lookup as a zero-cost backup for low-volume edge cases. Standalone Neural Networks should not be used for 1-hour fleet dispatching due to disproportionate operational overhead and lower variance capture.

3.3 Analysis using Reinforcement Learning

While the previous sections predict demand, this section turns to a decision problem at the level of a single vehicle: how should an electric vehicle (EV) be charged when the driver does not yet know how much energy the next day will require? We study this in a simulation. An EV arrives at home at 2 pm with between 5 and 15 kWh left in its 60 kWh battery and can charge for two hours, split into eight 15-minute slots. In each slot the charger runs at one of four levels (0, 7, 11, or 22 kW), matching common home wallbox and fast-charging powers. The next day's energy need is unknown while charging; we model it as normally distributed with a mean of 30 kWh and a standard deviation of 5 kWh. For the cost of charging, we employ an exponential formula of the power, which gives the following price for the charging levels: one slot costs \$1.02 at 7 kW, \$2.15 at 11 kW, but \$9.62 at 22 kW (Figure 42). This reflects that fast charging is disproportionately expensive in practice, through demand charges, conversion losses, and battery wear, and it creates the central issue of the problem: charging slowly is cheap but may not fill the battery in time. Finally, if the car leaves with less energy than the day turns out to require, a flat penalty of \$100 is incurred, representing the cost and inconvenience of a stranded driver. This setting can be translated into a Markov decision process (MDP). Figure 43 shows one episode of this MDP: the car arrives with a random charge, the charging loop repeats eight times with deterministic transitions, and only the final departure step branches, on the drawn demand.

Because we built the simulation ourselves, we know the complete model of this MDP: the transition rule and the demand distribution, which even gives the expected departure penalty in closed form. Dynamic programming computes the exact value of every action in every state, and with it the best achievable expected cost: \$39.10 per day, averaged over the random arrival charge. This is our reference metric. The learning agents, in contrast, never see the tariff, the transition rule, or the demand distribution; they only observe the current state, choose an action, and receive a reward.

Within model-free control, the structure of the problem guides the choice of method. The task is episodic with short episodes that always terminate after eight steps, which makes Monte Carlo control applicable and attractive. Q-learning is the temporal-difference alternative: it updates after every single step, moving the value of a decision toward the observed step reward plus its own current estimate of the best continuation. Both agents explore with an epsilon-greedy strategy, taking a random action 30 % of the time and their currently best-known action otherwise, over 80,000 simulated episodes. Since the state space is small and discrete (241 battery levels times 8 time slots, with 4 actions), both methods store their value estimates in an exact table. For the same reason we avoid a Deep Q-Network: replacing the table with a neural network is necessary only when the state space is too large or continuous to enumerate, and here it would add approximation error, hyperparameter tuning, and training instability without any benefit.

Figure 44 shows both learning curves on one axis, measured identically: the expected cost of each agent's current best policy during training. Monte Carlo is significantly faster, reaching close to its final quality after a few thousand days, while Q-learning needs tens of thousands.

Both agents discover the same, quite intuitive strategy: charge steadily at medium power towards roughly 37 to 40 kWh, a safety buffer of about one and a half standard deviations above the average demand, and use the expensive fast-charging level only on days that start with a nearly empty battery. Charging merely to the average demand would mean running short every second day, while filling the battery completely would waste money on energy that is rarely needed; the learned buffer balances the extra charging cost against the risk of the penalty. In money terms, Monte Carlo reaches an expected daily cost of \$41.03 and Q-learning \$41.06, both

within about 5 % of the optimal \$39.10. Figure 45 compares the learned decision rules with the optimal one across all situations.

The practical message is that a smart charger can learn a good charging strategy purely from experience. It was never told the electricity tariff, the demand distribution, or the penalty; it inferred the trade-off from outcomes alone. The learned behaviour also translates into simple advice: charge steadily at moderate power and keep a safety buffer above the expected need.

4 Conclusion

This project develops a framework to help a car manufacturer to enter the US ride-hailing market by answering one core question: where, when, and how much demand will occur. Using Chicago taxi trips, we approached it in three stages: a descriptive analysis of taxi patterns, predictive models that forecast demand, and a reinforcement-learning study of EV charging.

A lesson across the project was that careful data preparation and analysis matter as much as the modeling itself. Cleaning and aggregating the trips revealed, for example, that the spatial data is less precise than it first appears (mostly community area level instead of exact coordinates) and that weather has a surprisingly low impact on demand, which may be counterintuitive. The descriptive analysis then showed that Chicago’s taxi demand is heavily concentrated in the downtown core and around O’Hare Airport and has only small seasonal variation. This regularity is itself a valuable finding and it explains why location features such as points of interest increase demand far more than volatile inputs like weather.

The Prediction stage confirmed that this structure can be turned into reliable forecasts. Both support vector machines and neural networks outperformed a naive historical-average benchmark, and the two families performed similarly to one another. Because their accuracy was comparable, the choice between them came down to practical considerations rather than raw performance: the SVM trains quickly, behaves deterministically, and tracks demand surges slightly more faithfully, which is why we recommend it as main model. Just as important was the learning that the most complex model is not always the best, simpler configurations often matched or even beat more complex ones. This recommendation comes with two caveats. It rests on one design decision: aggregating trips into area-and-time buckets before the models saw them, and a different starting point could shift the balance. It also compares only two model families. A continued engagement would test others, in particular hybrid designs such as an SVM producing a first-pass estimate with a neural network on top as a “reasoner” that refines it.

A Reinforcement Learning study addressed the operational charging strategy at the level of a single vehicle. We showed that a smart charger can learn a good charging policy purely from experience: charging steadily at moderate power, keeping a buffer above the expected daily need. Both learning agents came within roughly 5% of the mathematically optimal cost. Intelligent charging, does not require perfect information, a self-learning agent can find the almost best achievable cost on its own.

Taken together, the three stages give the client a data-driven foundation for its fleet deployment, rebalancing, and charging decisions as it enters the market. The key takeaway of our study is: start with the simplest projection of reality, which does not even need to involve machine learning, and increase complexity gradually, always keeping an eye on cost and desired precision. For this project specifically, we suggest a SVM model employing POI data and not using weather data at an hourly, hexagon-level resolution. In a future examination smart-city sensor counts of how many people pass through an area, or Chicago tourism, demographic and event data would likely be a good source of input for more elaborate models.

5 Appendix A: Figures

Columns (23)			
Column Name	Description	API Field Name	Data Type
Trip ID	A unique identifier for the trip.	trip_id	Text
Taxi ID	A unique identifier for the taxi.	taxi_id	Text
Trip Start Timestamp	When the trip started, rounded to the nearest 15 minutes.	trip_start_timestamp	Floating Timestamp
Trip End Timestamp	When the trip ended, rounded to the nearest 15 minutes.	trip_end_timestamp	Floating Timestamp
Trip Seconds	Time of the trip in seconds.	trip_seconds	Number
Trip Miles	Distance of the trip in miles.	trip_miles	Number
Pickup Census Tract	The Census Tract where the trip began. For privacy, this Census Tract is not shown for some trips. This column often will be blank for locations outside Chicago.	pickup_census_tract	Text
Dropoff Census Tract	The Census Tract where the trip ended. For privacy, this Census Tract is not shown for some trips. This column often will be blank for locations outside Chicago.	dropoff_census_tract	Text
Pickup Community Area	The Community Area where the trip began. This column will be blank for locations outside Chicago.	pickup_community_area	Number
Dropoff Community Area	The Community Area where the trip ended. This column will be blank for locations outside Chicago.	dropoff_community_area	Number
Fare	The fare for the trip.	fare	Number
Tips	The tip for the trip. Cash tips generally will not be recorded.	tips	Number
Tolls	The tolls for the trip.	tolls	Number
Extras	Extra charges for the trip.	extras	Number
Trip Total	Total cost of the trip, the total of the previous columns.	trip_total	Number
Payment Type	Type of payment for the trip.	payment_type	Text
Company	The taxi company.	company	Text
Pickup Centroid Latitude	The latitude of the center of the pickup census tract or the community area if the census tract has been hidden for privacy. This column often will be blank for locations outside Chicago. Read less	pickup_centroid_latitude	Number
Pickup Centroid Longitude	The longitude of the center of the pickup census tract or the community area if the census tract has been hidden for privacy. This column often will be blank for locations outside Chicago. Read less	pickup_centroid_longitude	Number
Pickup Centroid Location	The location of the center of the pickup census tract or the community area if the census tract has been hidden for privacy. This column often will be blank for locations outside Chicago. Read less	pickup_centroid_location	Point
Dropoff Centroid Latitude	The latitude of the center of the dropoff census tract or the community area if the census tract has been hidden for privacy. This column often will be blank for locations outside Chicago. Read less	dropoff_centroid_latitude	Number
Dropoff Centroid Longitude	The longitude of the center of the dropoff census tract or the community area if the census tract has been hidden for privacy. This column often will be blank for locations outside Chicago. Read less	dropoff_centroid_longitude	Number
Dropoff Centroid Location	The location of the center of the dropoff census tract or the community area if the census tract has been hidden for privacy. This column often will be blank for locations outside Chicago. Read less	dropoff_centroid_location	Point

Figure 1: Raw Chicago Taxi Trip Data Column Descriptions

```

RangeIndex: 17568 entries, 0 to 17567
Data columns (total 15 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   date                                  17568 non-null  datetime64[ms, UTC]
1   temperature_2m                       17568 non-null  float32
2   relative_humidity_2m                 17568 non-null  float32
3   apparent_temperature                 17568 non-null  float32
4   precipitation                         17568 non-null  float32
5   rain                                 17568 non-null  float32
6   snowfall                             17568 non-null  float32
7   snow_depth                           17568 non-null  float32
8   surface_pressure                     17568 non-null  float32
9   cloud_cover                          17568 non-null  float32
10  wind_speed_10m                       17568 non-null  float32
11  wind_speed_100m                      17568 non-null  float32
12  is_day                               17568 non-null  float32
13  sunshine_duration                    17568 non-null  float32
14  direct_radiation                     17568 non-null  float32
dtypes: datetime64[ms, UTC](1), float32(14)

```

Figure 2: Weather Data Columns

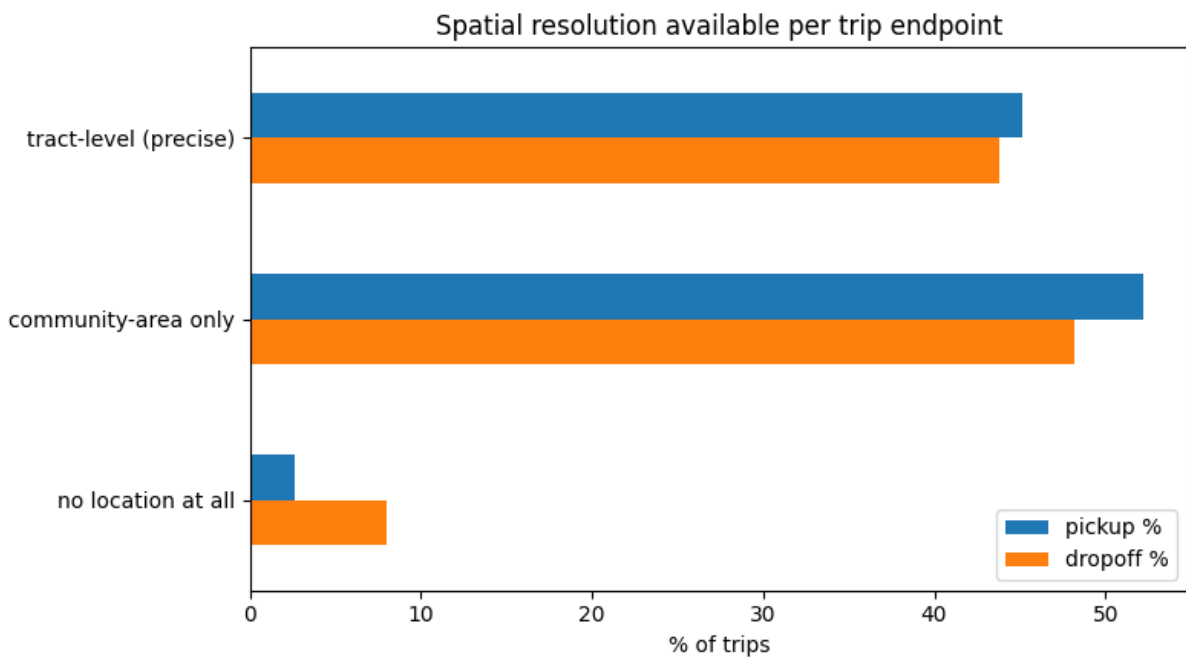


Figure 3: Spatial resolution available per trip endpoint

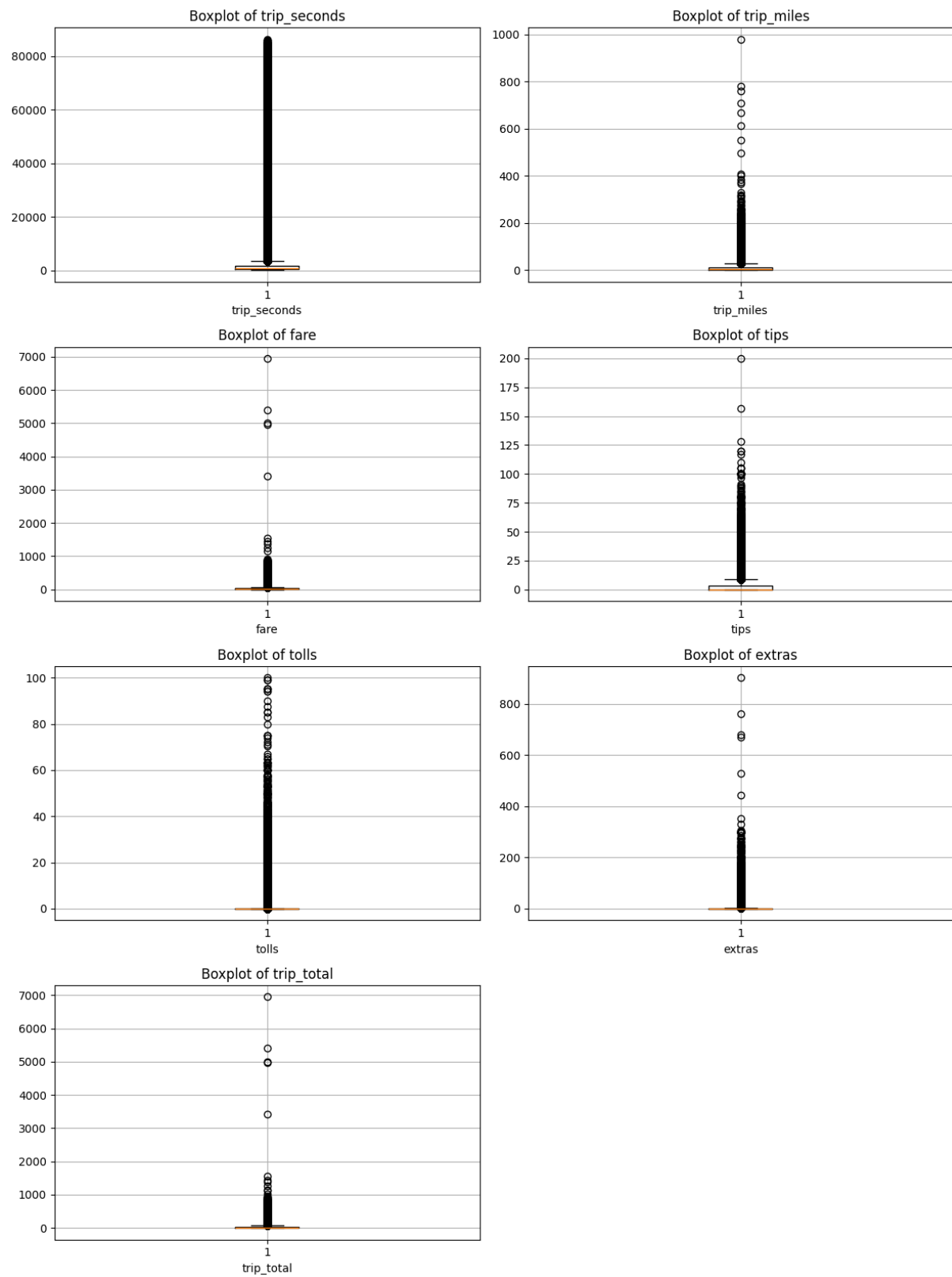


Figure 4: Boxplot of taxi trip attributes to detect outliers

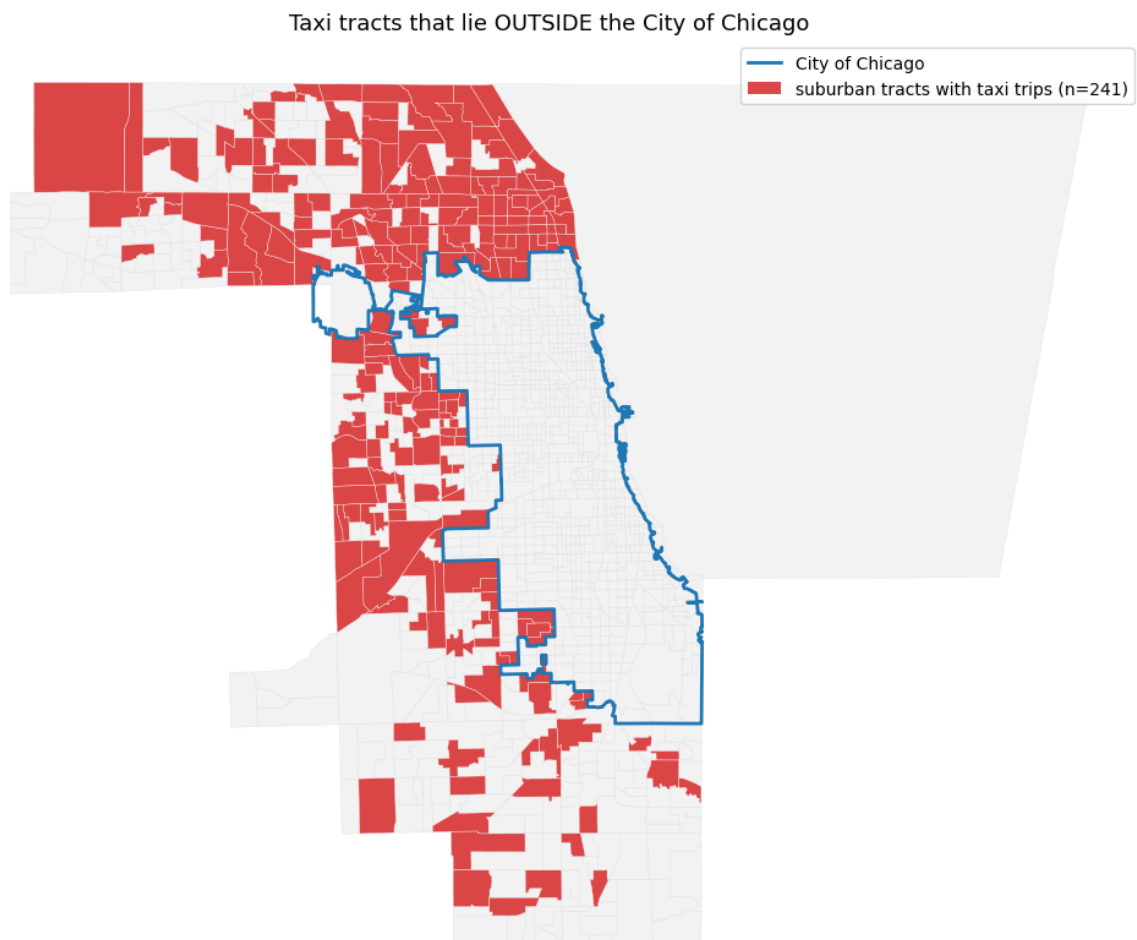


Figure 5: census tracts with trips outside of Chicago

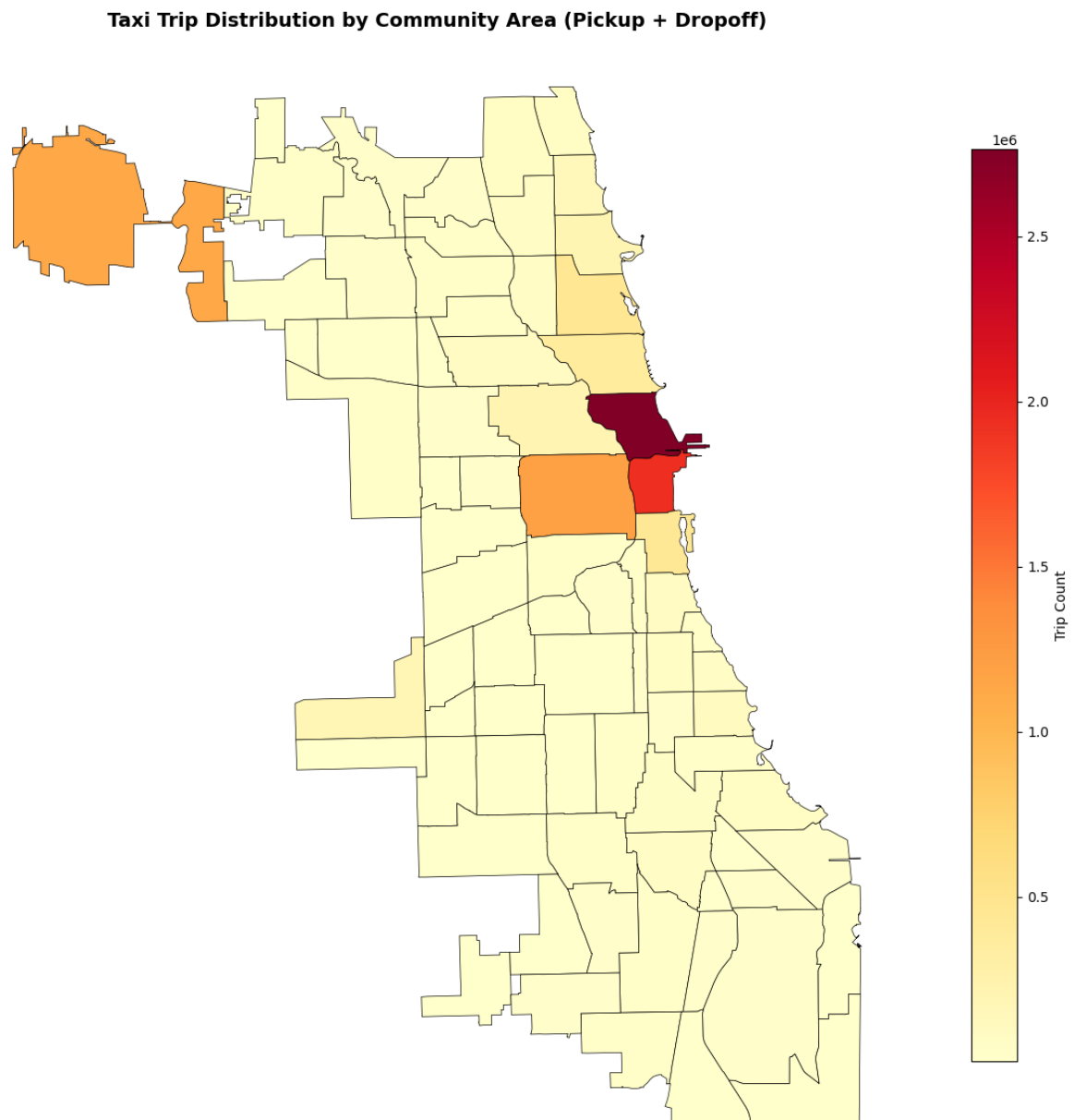


Figure 6: Trip distribution plots at the community area level

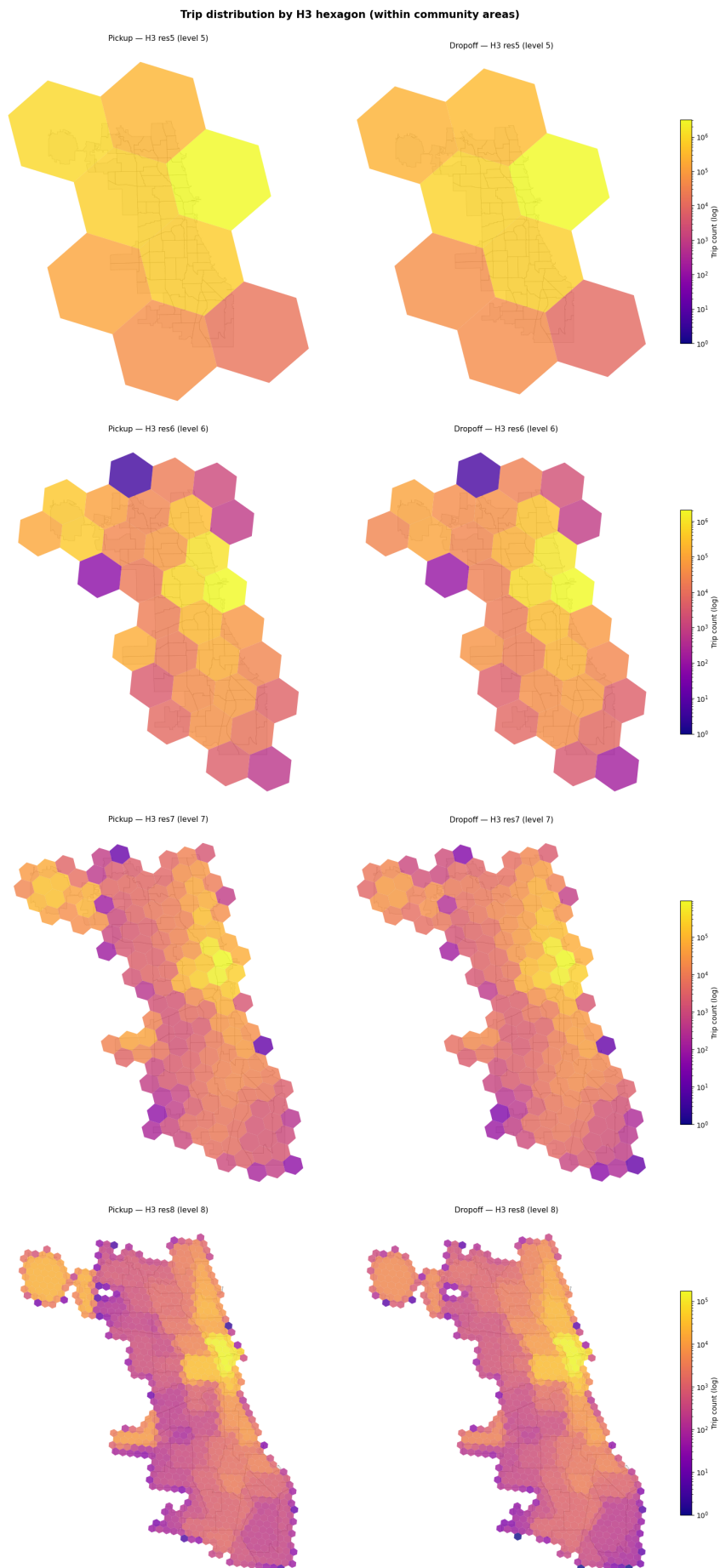


Figure 7: Trip distribution plots for H3 hexagons at multiple levels

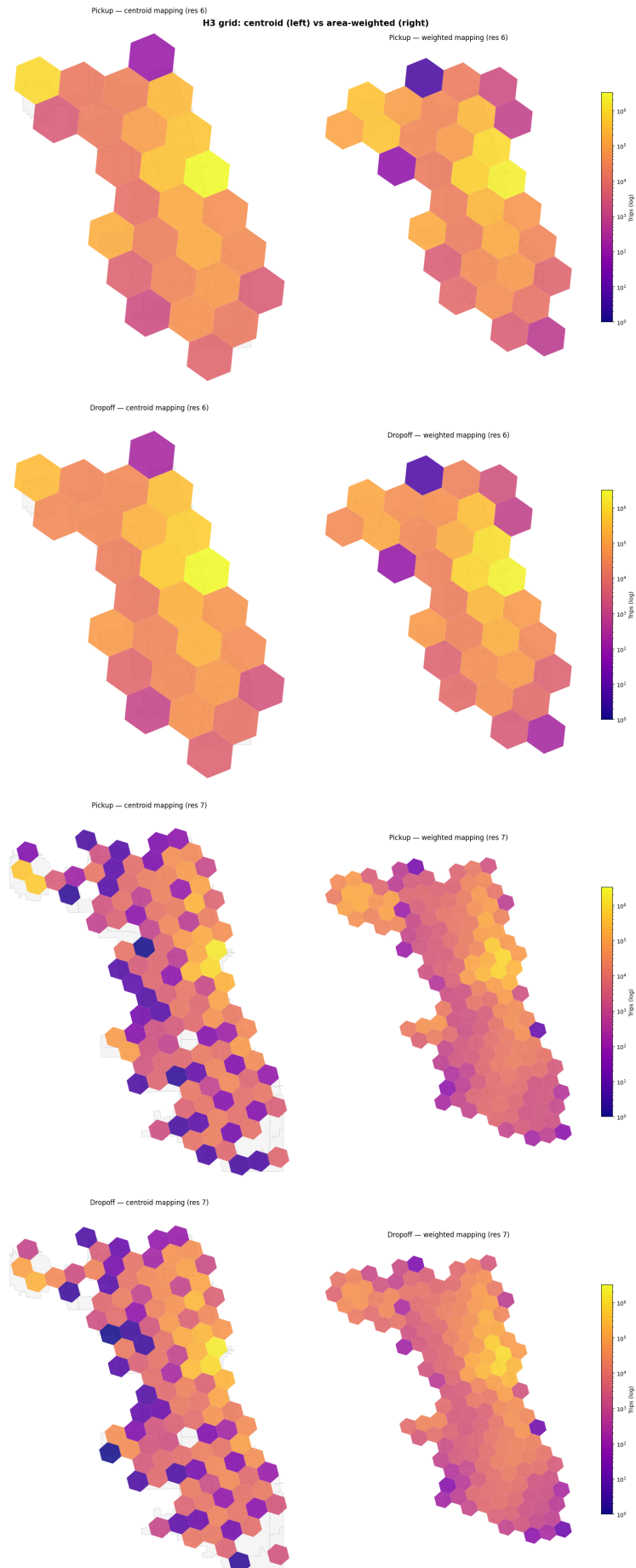


Figure 8: Trip distribution for H3 Hexagons: centroids vs area-weighted

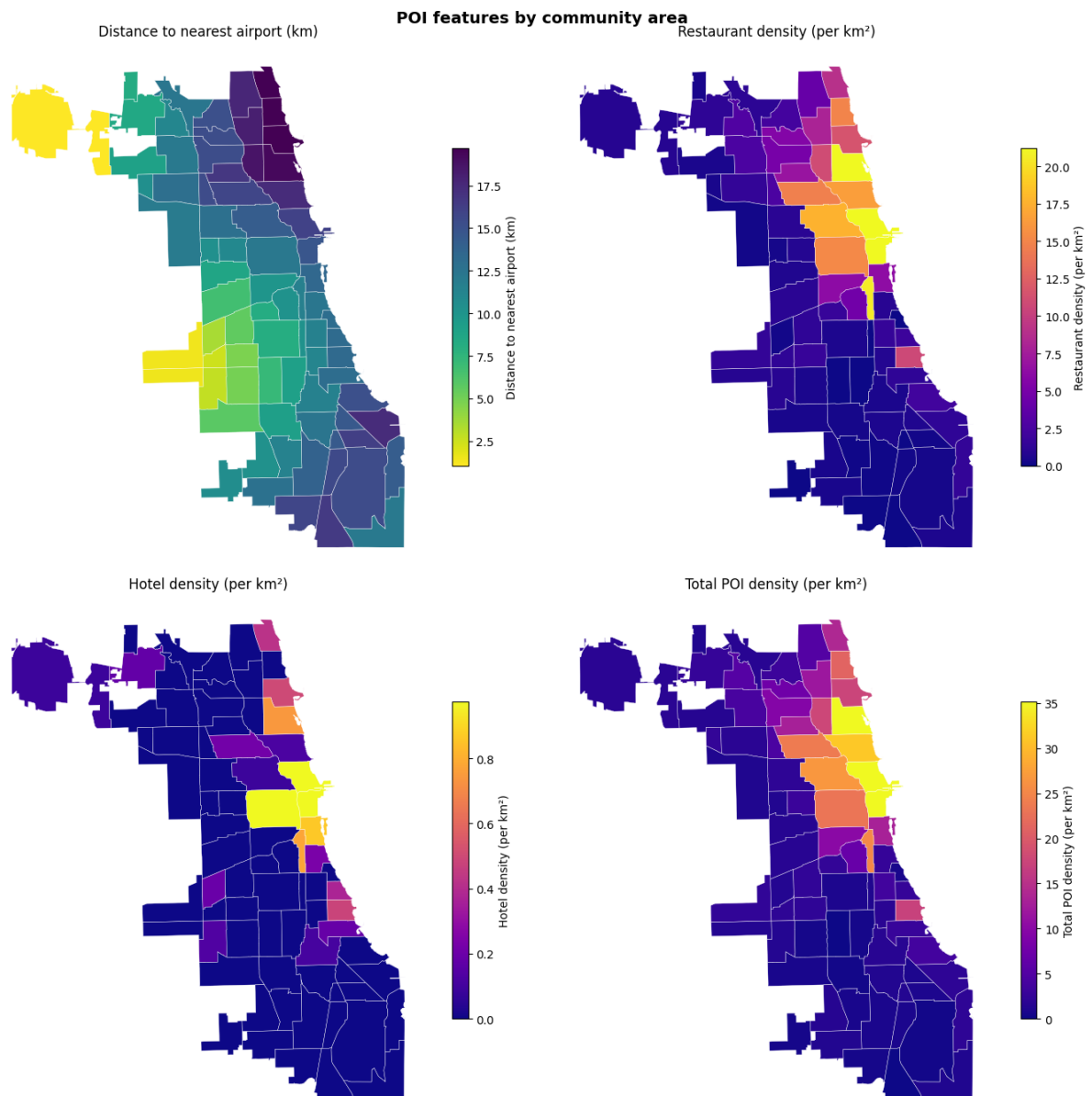


Figure 9: POI features by community area: distance to nearest airport, restaurant density, hotel density, and total POI density.

Trip distribution by H3 hexagon

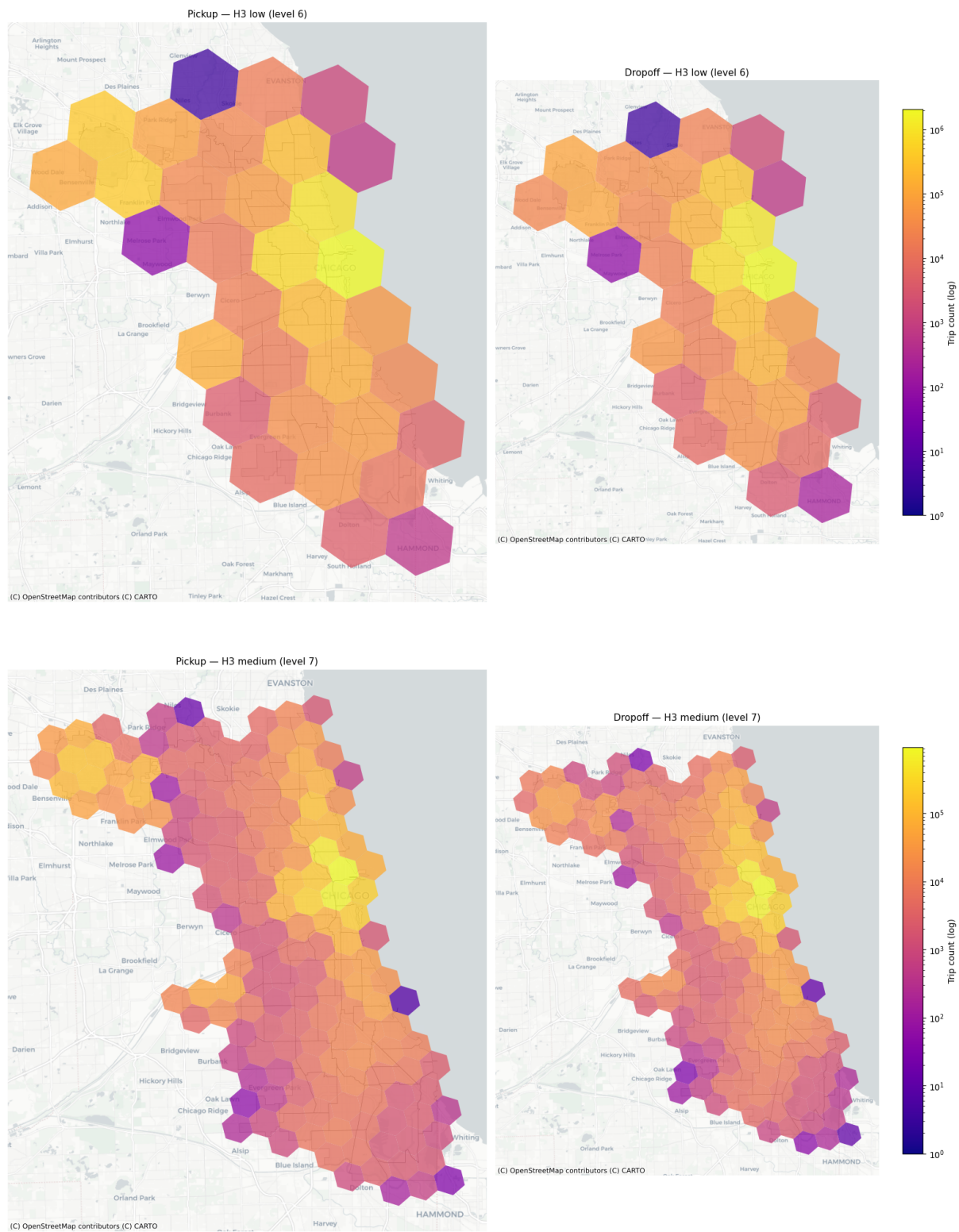


Figure 10: Trip distribution by H3 hexagon

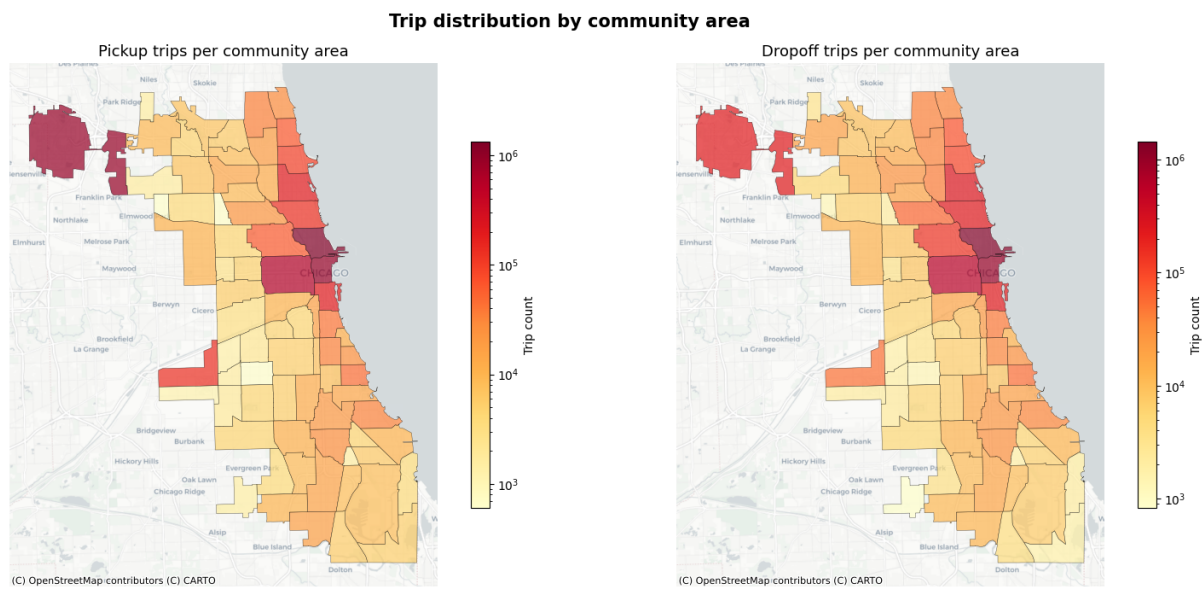


Figure 11: Trip distribution by community area

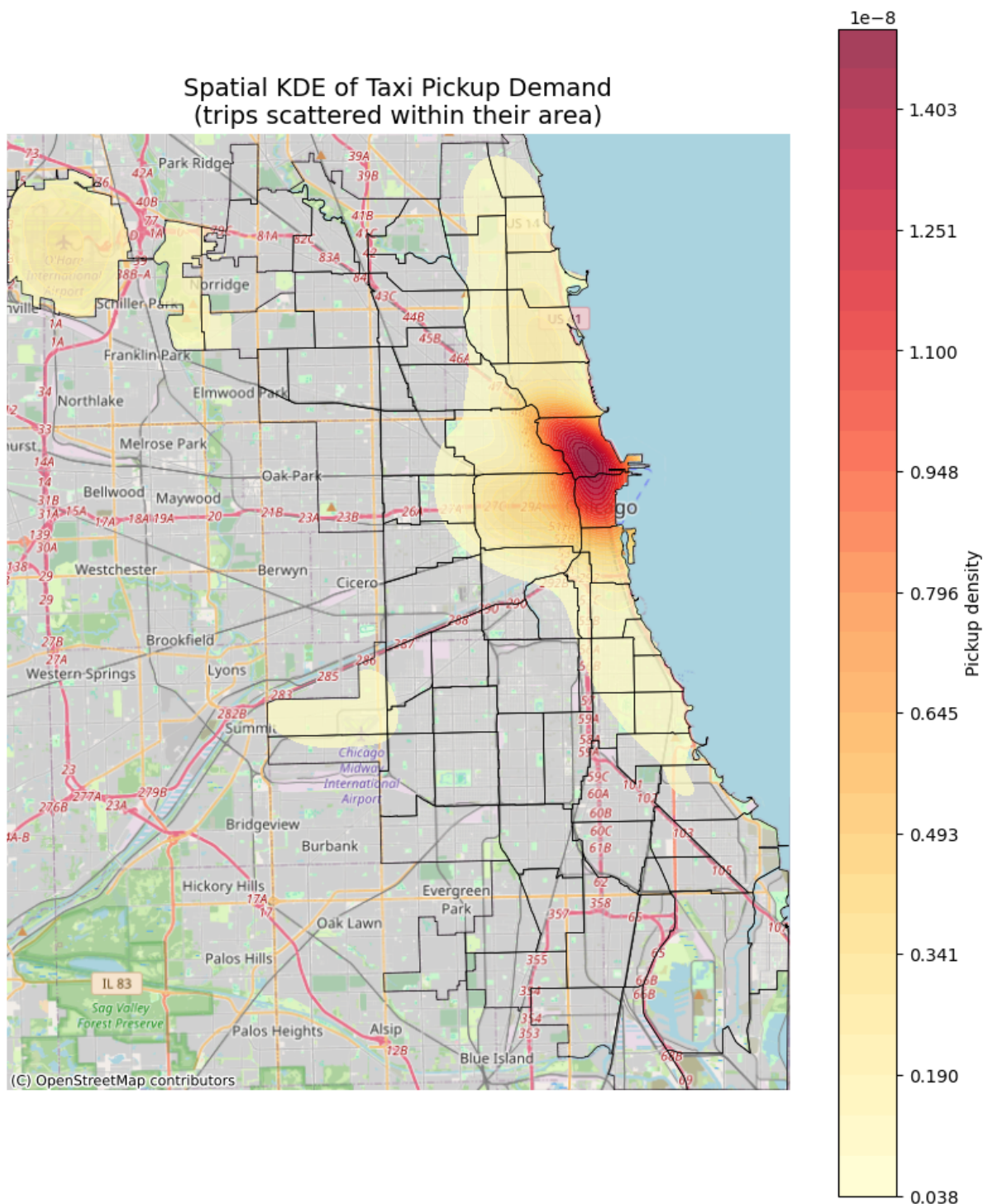


Figure 12: Spatial KDE of Taxi Pickup Demand

Spatial KDE of Taxi Pickup Demand by Time of Day
(trips scattered within their area)

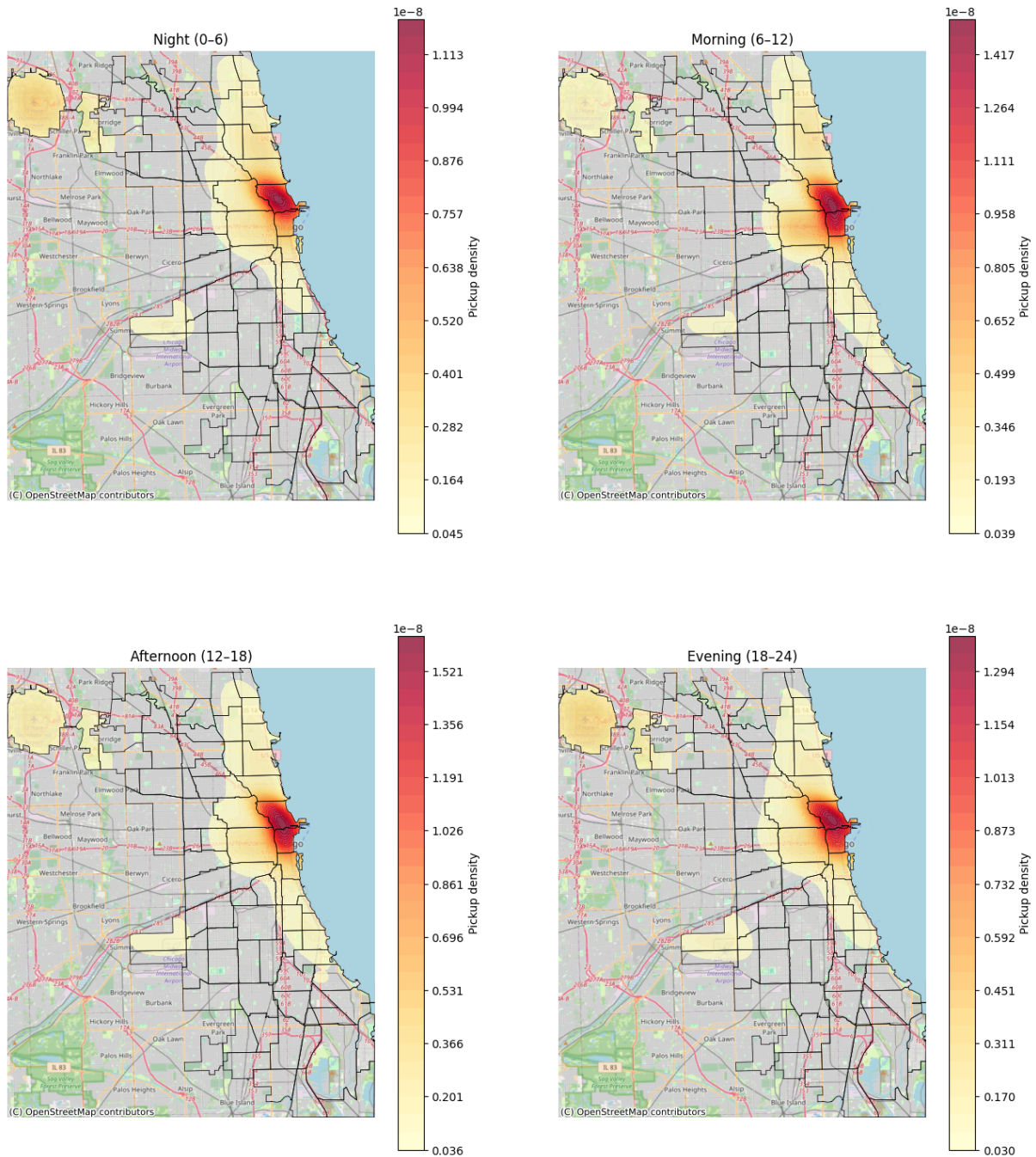


Figure 13: Spatial KDE of Taxi Pickup Demand by Time of Day

Avg Price by H3 hexagon

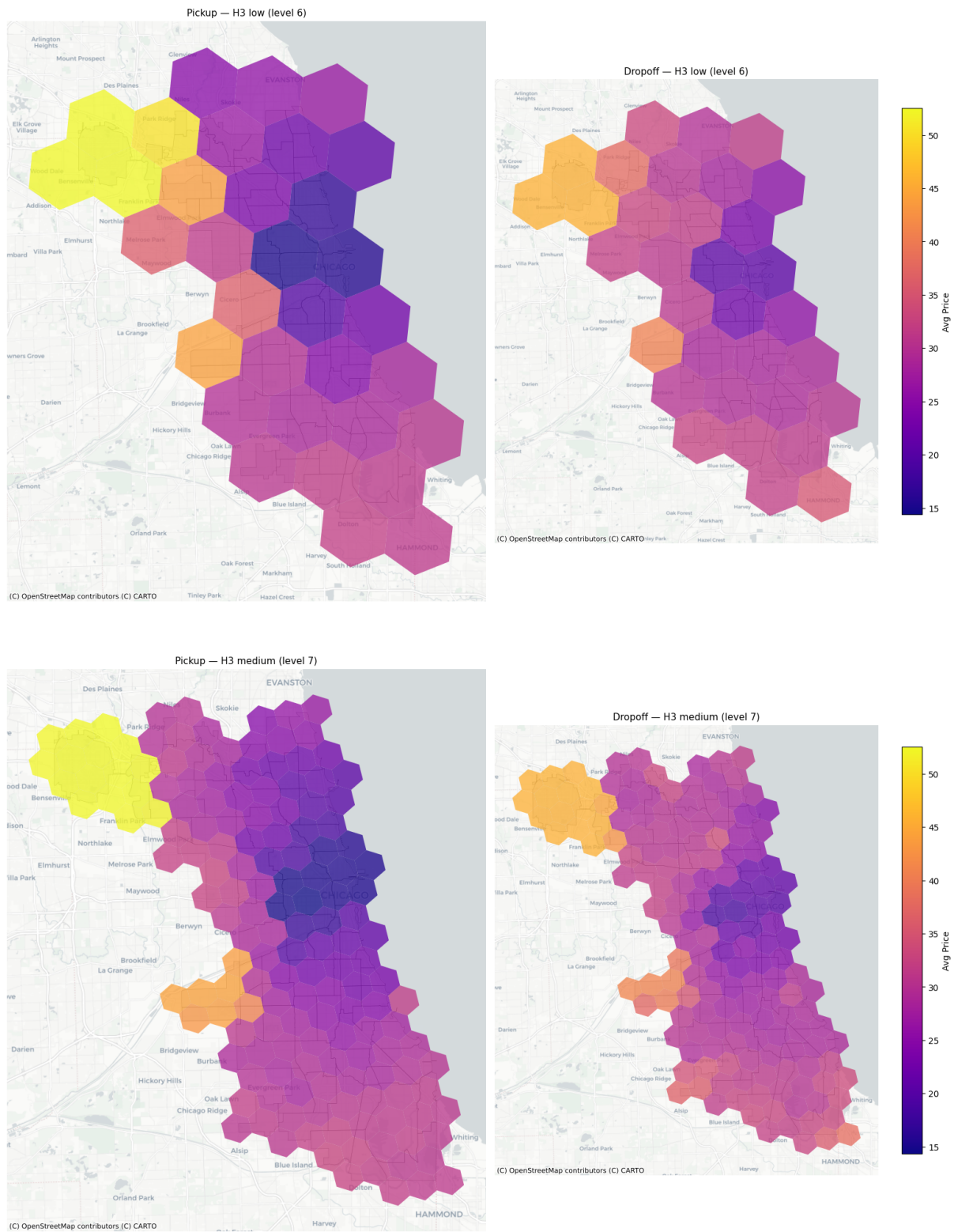


Figure 14: Average price by H3 hexagon

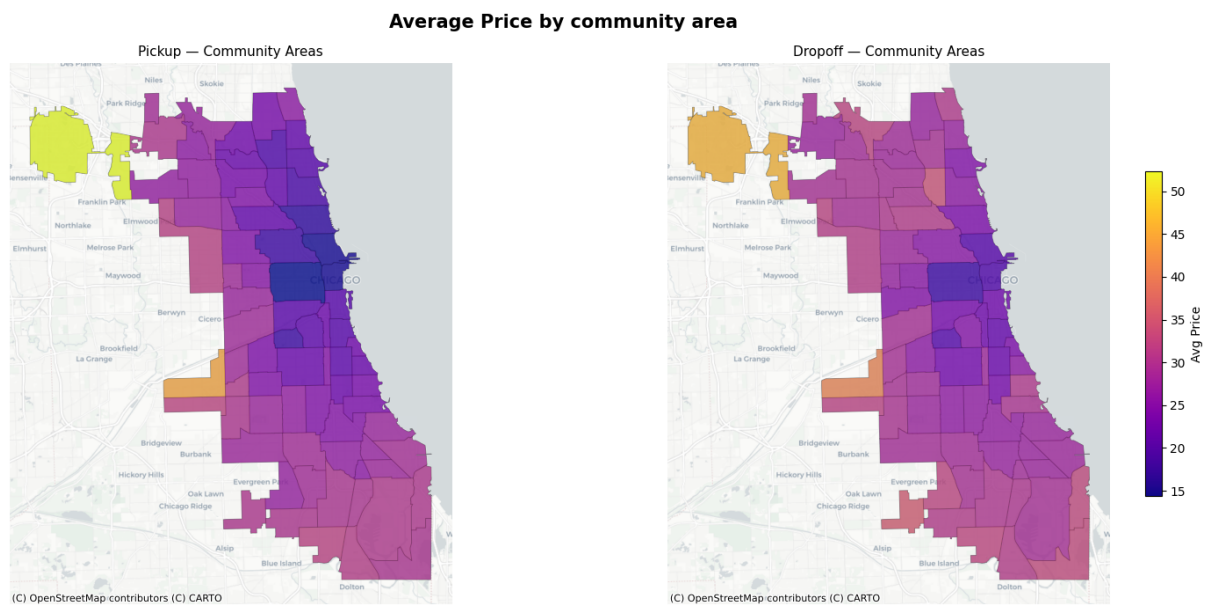


Figure 15: Average price by community area

Trip Miles by H3 hexagon

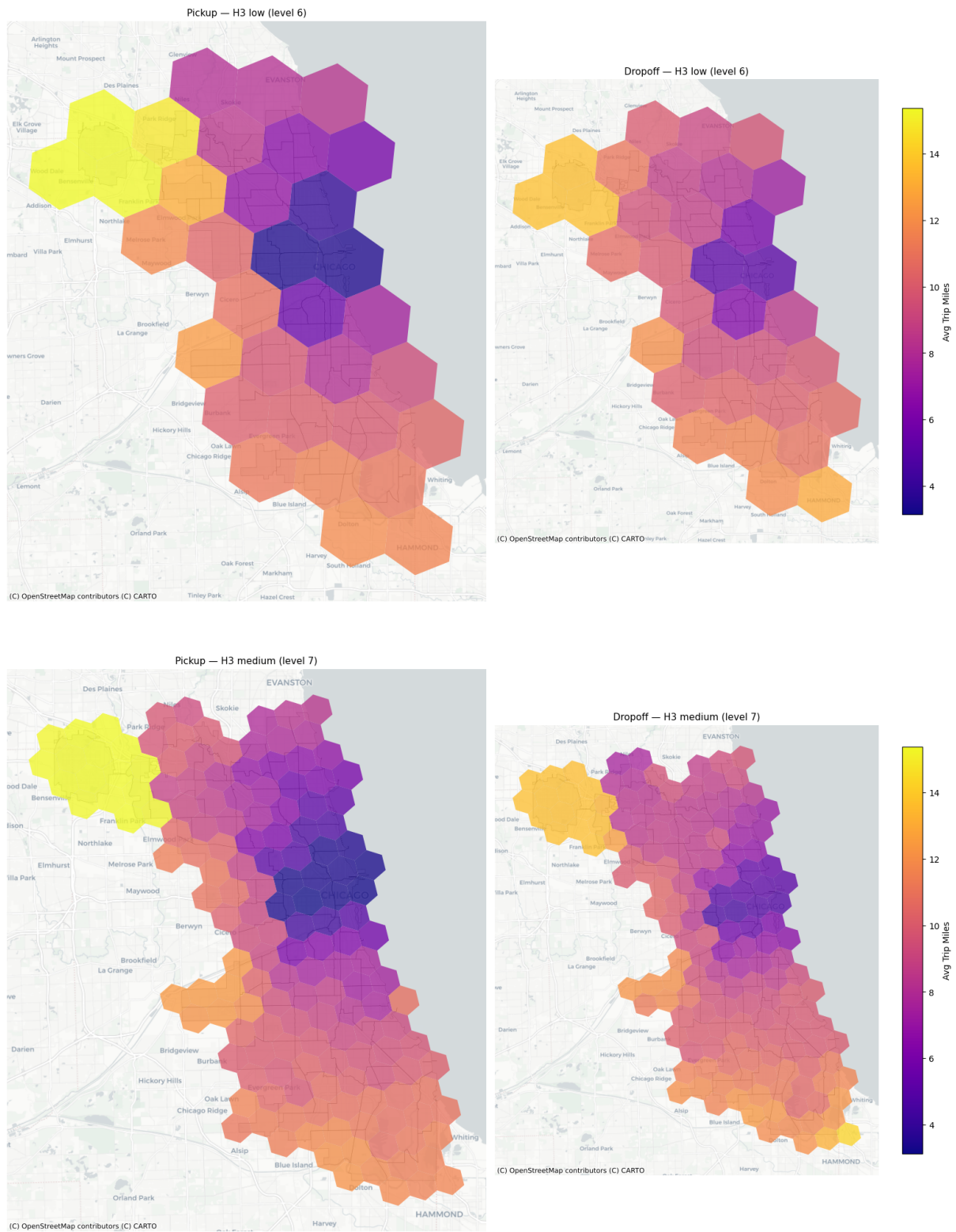


Figure 16: Average trip miles by H3 hexagon

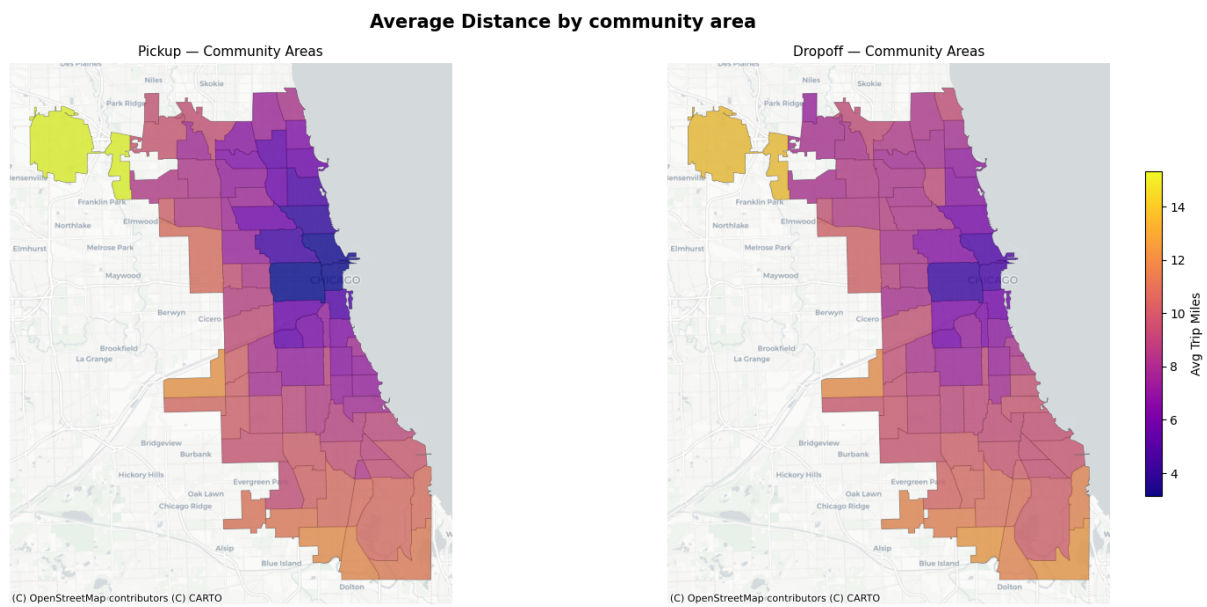


Figure 17: Average trip miles by community area

Trip Duration by H3 hexagon

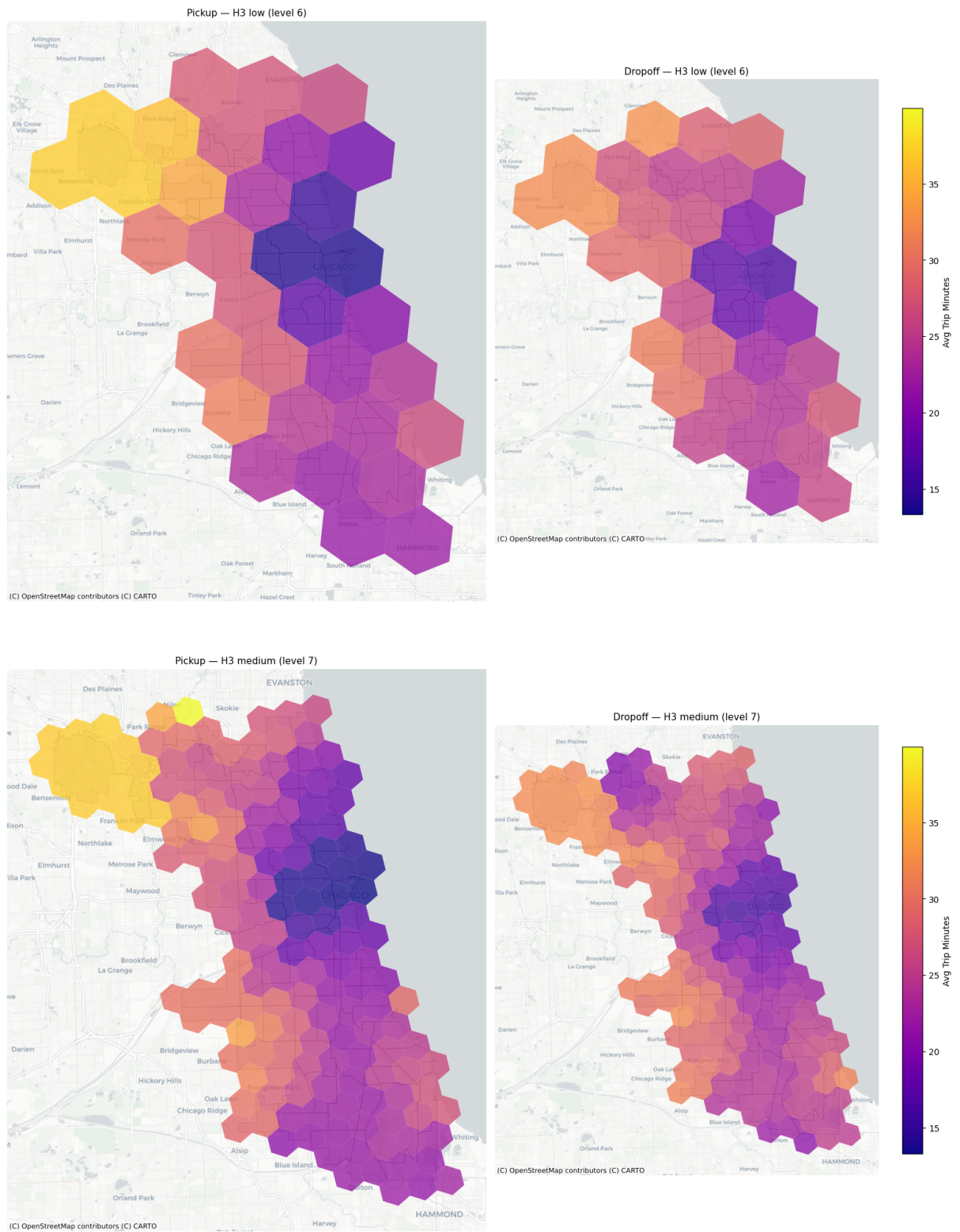


Figure 18: Trip Duration by H3 hexagon

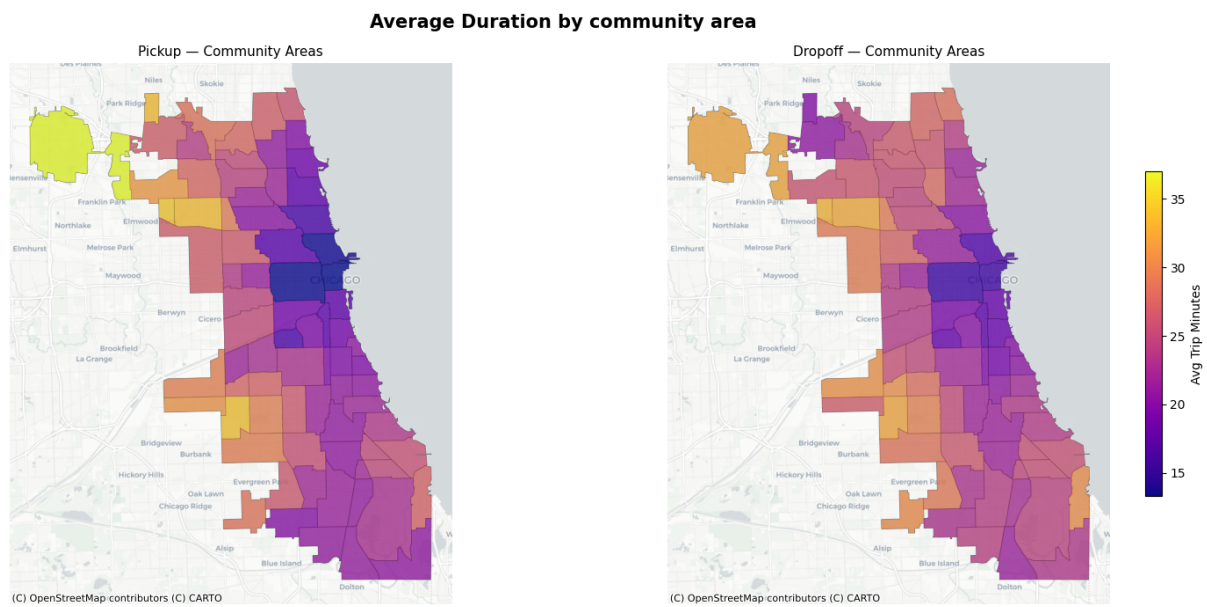


Figure 19: Trip Duration by community area

Idle Time by H3 hexagon

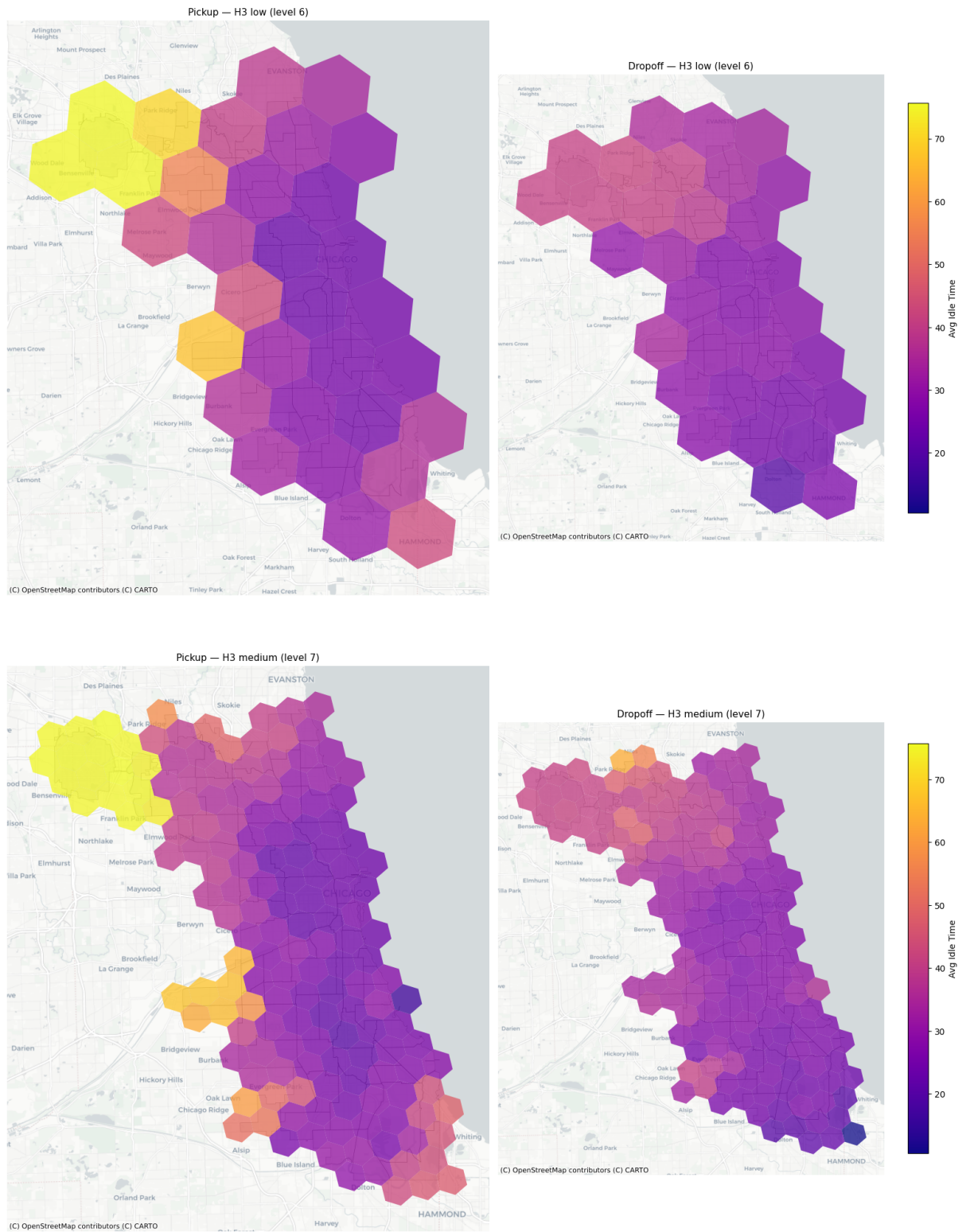


Figure 20: Idle Time by H3 hexagon

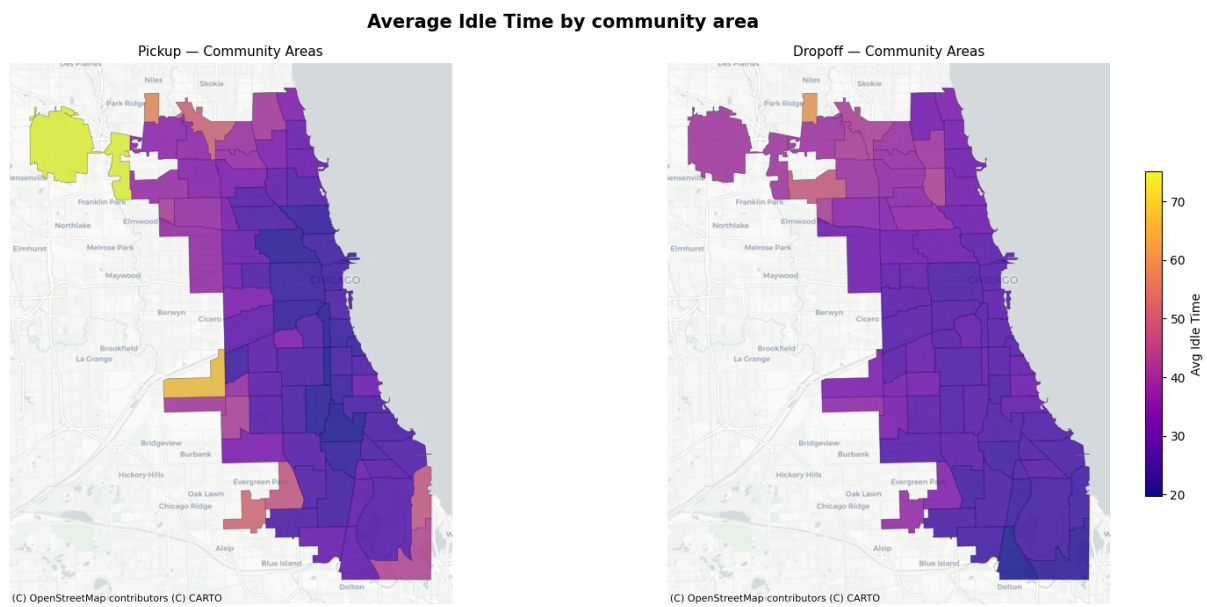


Figure 21: Idle Time by community area

Tip Rate by H3 hexagon

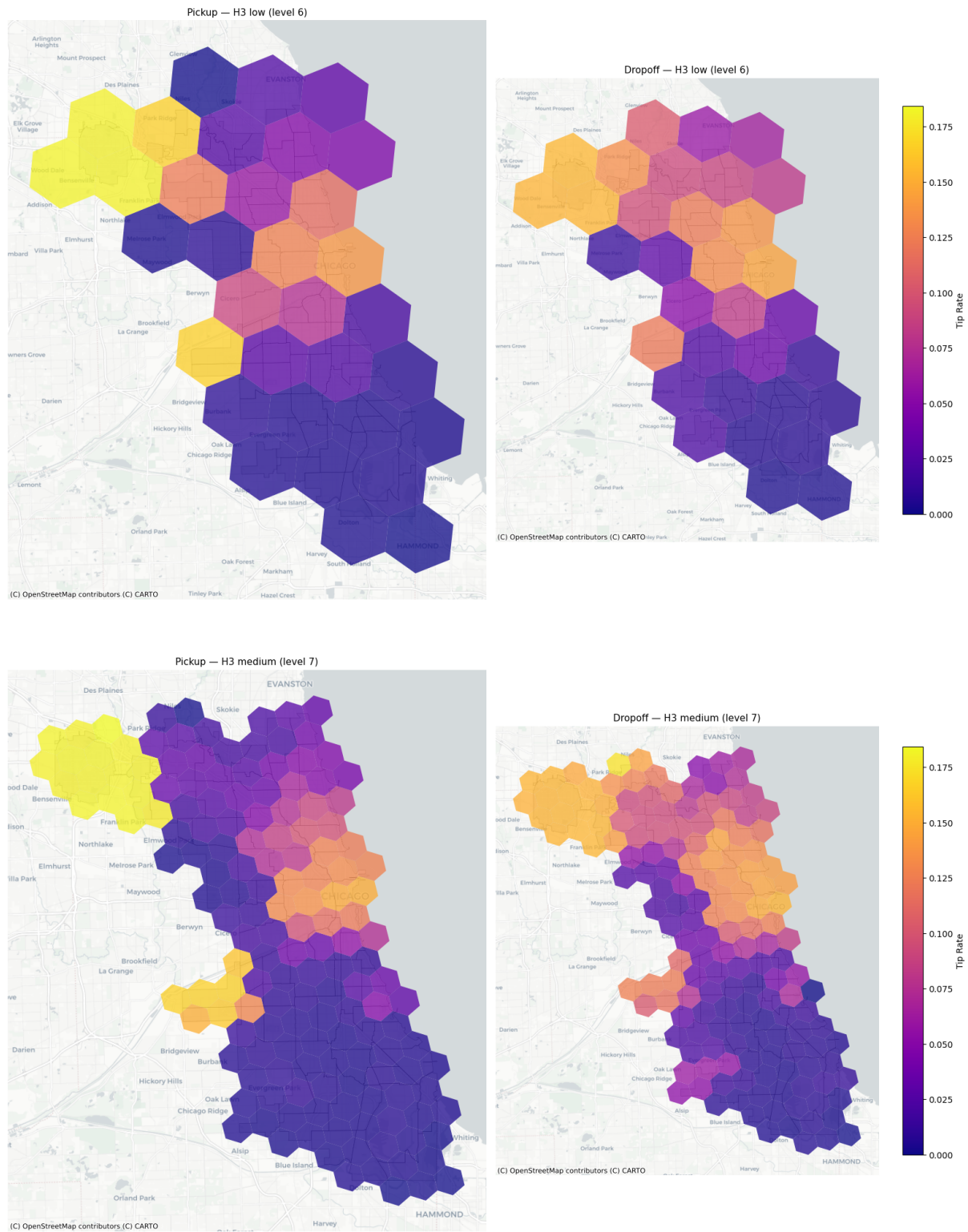


Figure 22: Tip Rate by H3 hexagon

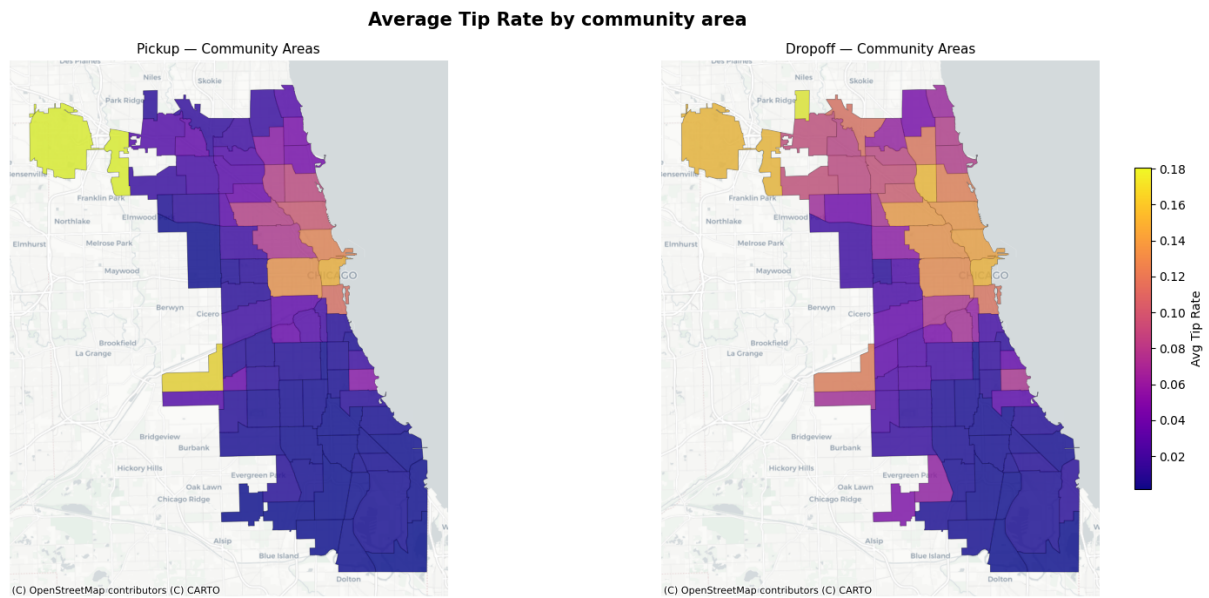


Figure 23: Tip Rate by community area

Trip starting time by H3 hexagon

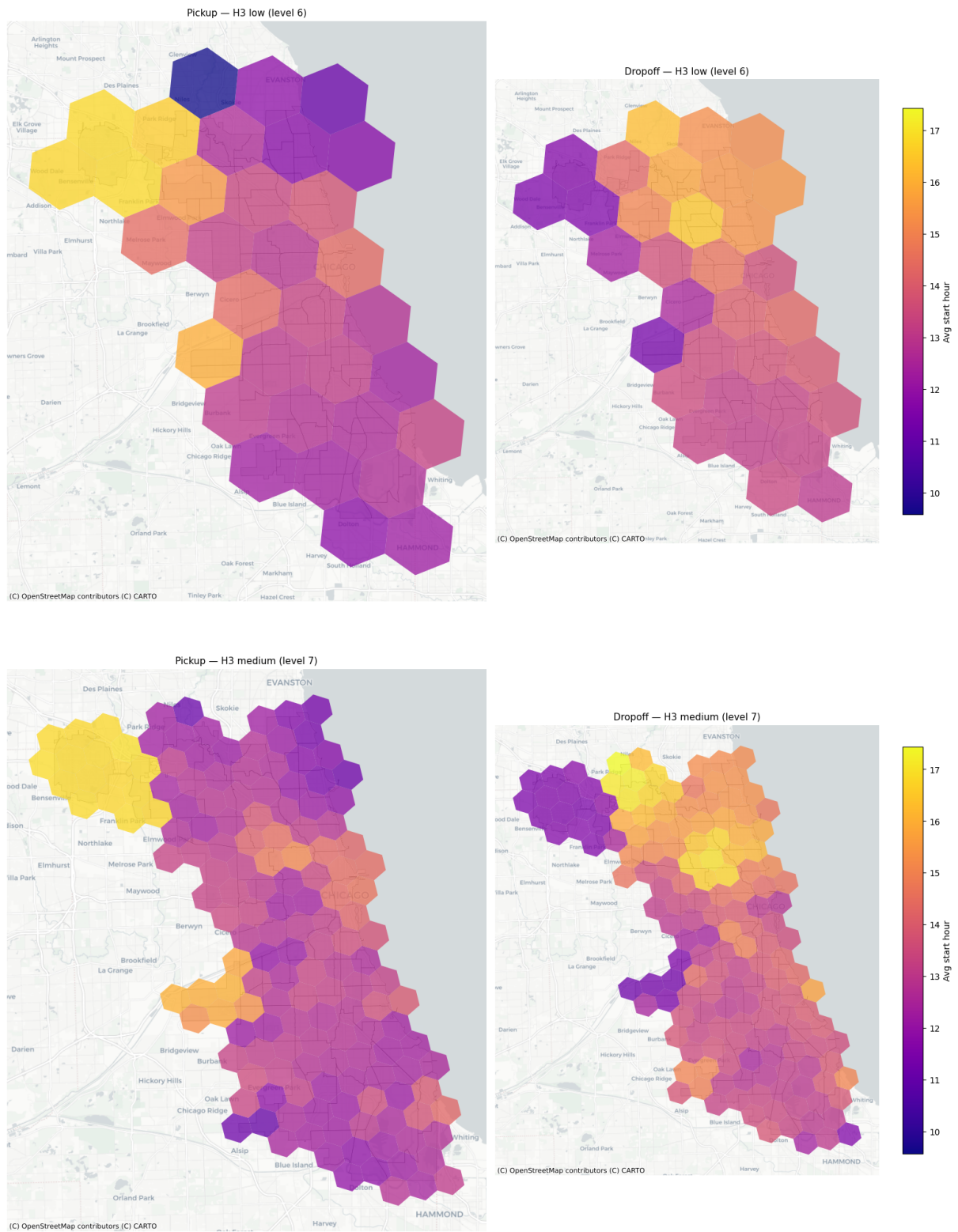


Figure 24: Start Time by H3 hexagon

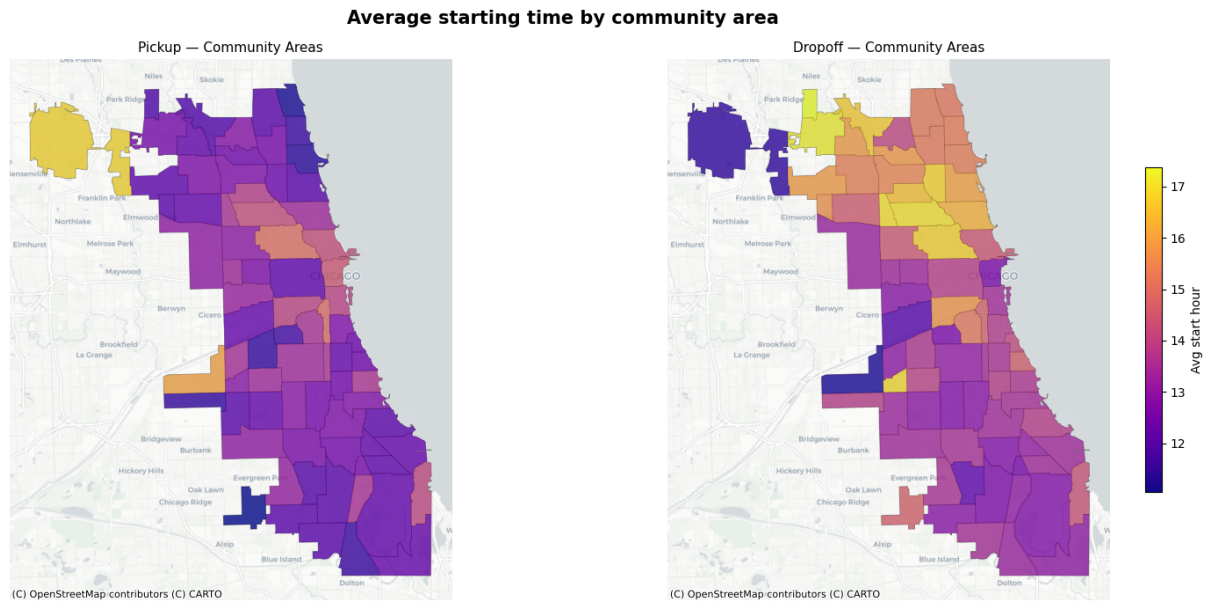


Figure 25: Start Time by community area

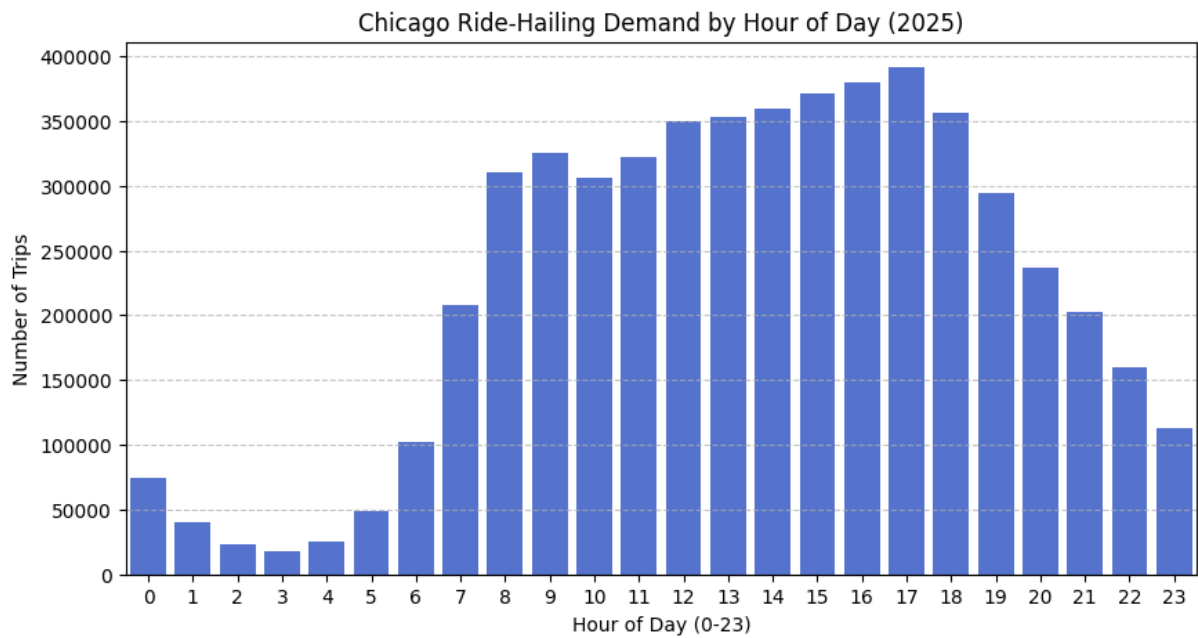


Figure 26: Chicago Ride-Hailing Demand by Hour of Day

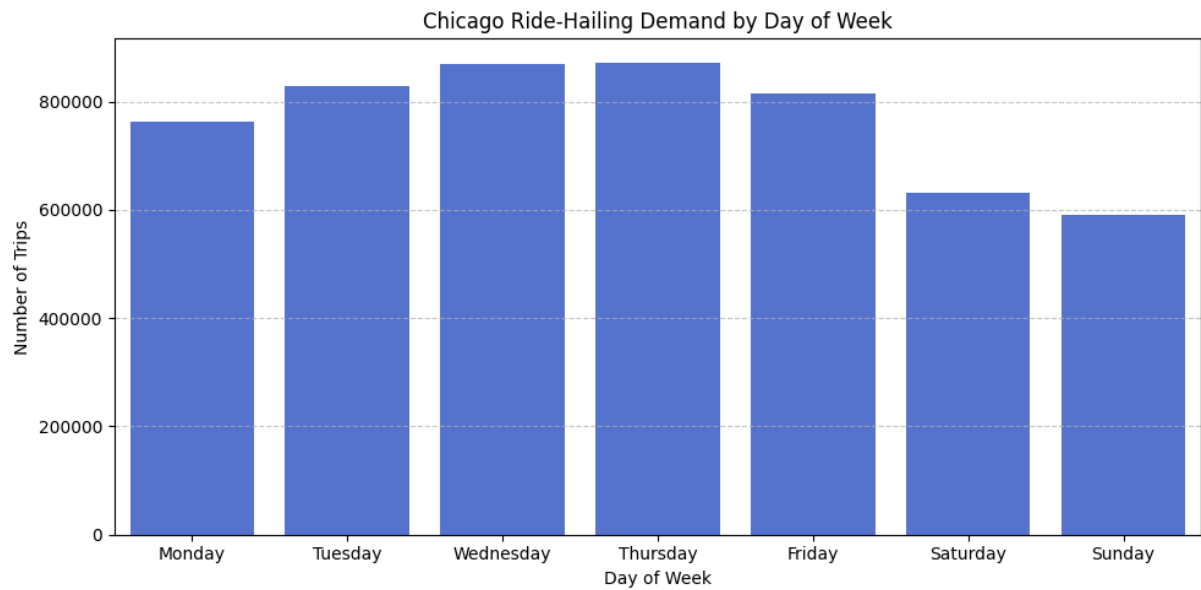


Figure 27: Chicago Ride-Hailing Demand by Day of Week

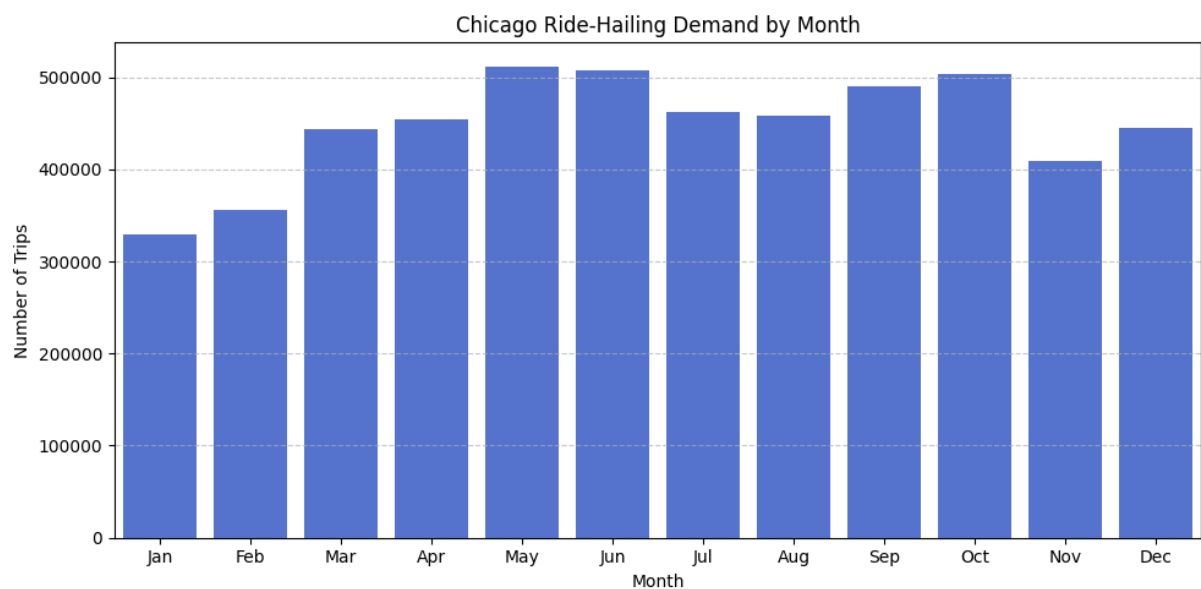


Figure 28: Chicago Ride-Hailing Demand by Month

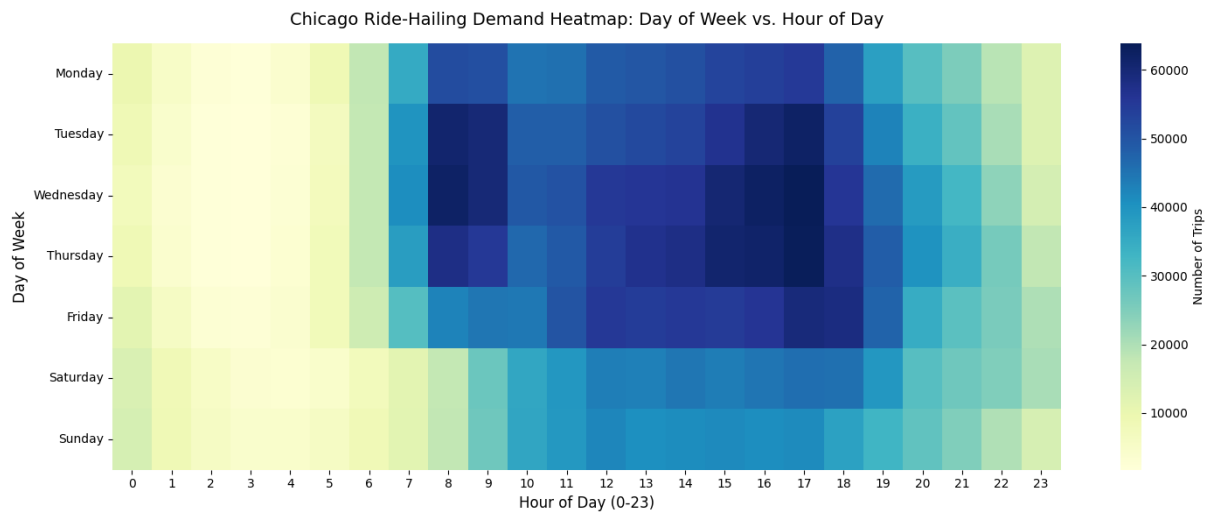


Figure 29: Chicago Ride-Hailing Demand Heatmap: Day of Week vs Hour of Day

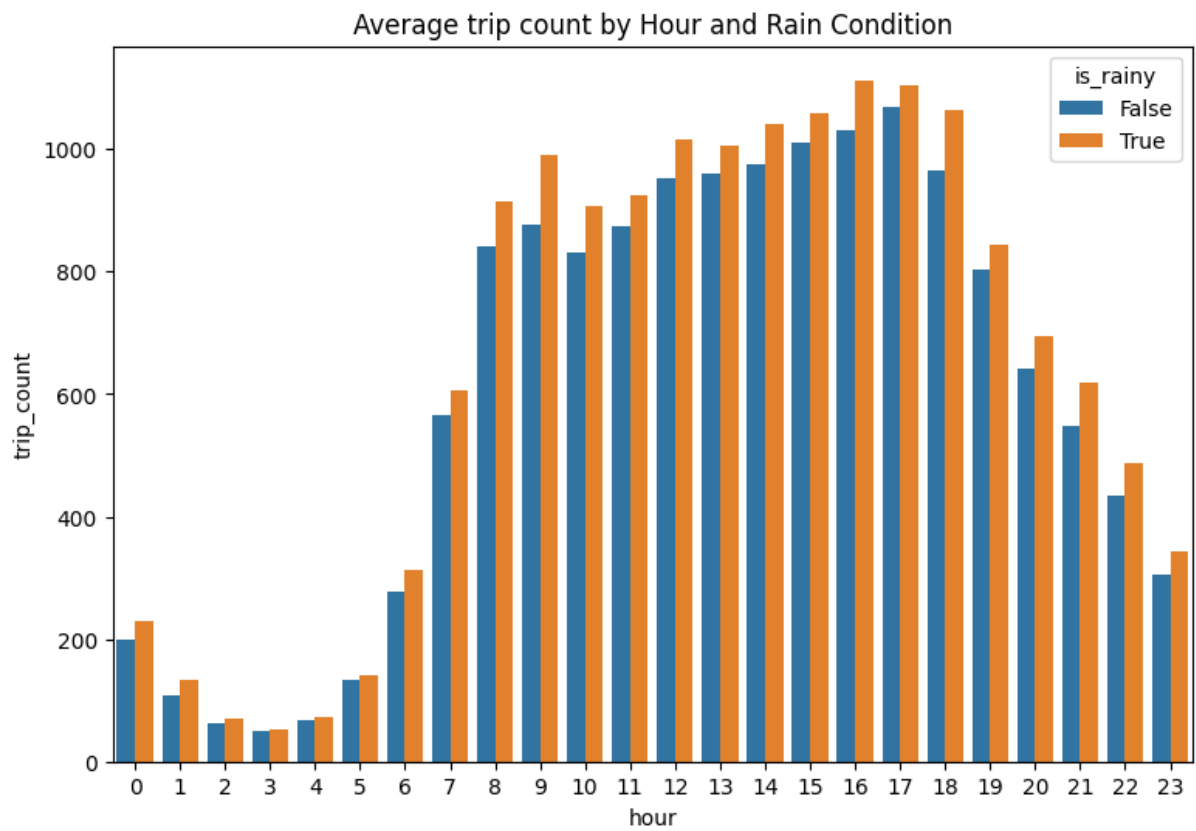


Figure 30: Average trip count by Hour and Rain Condition

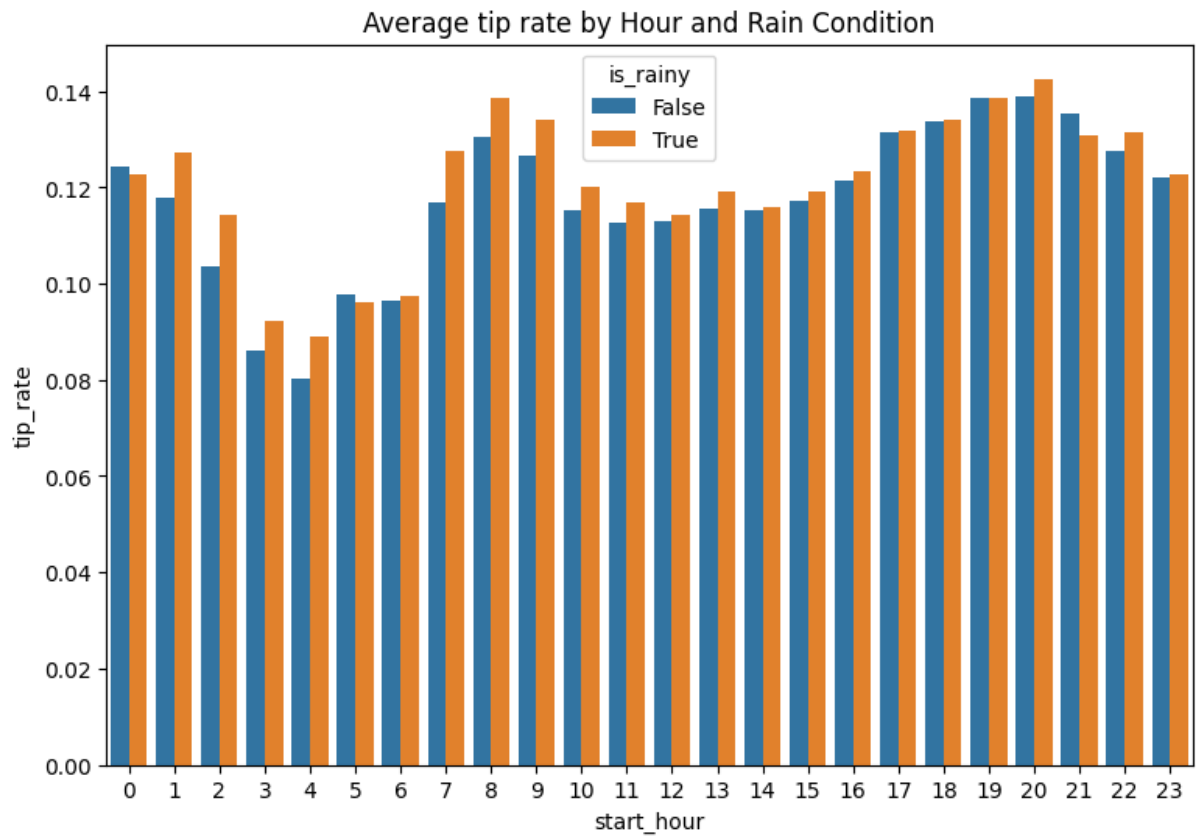


Figure 31: Average tip rate by Hour and Rain Condition

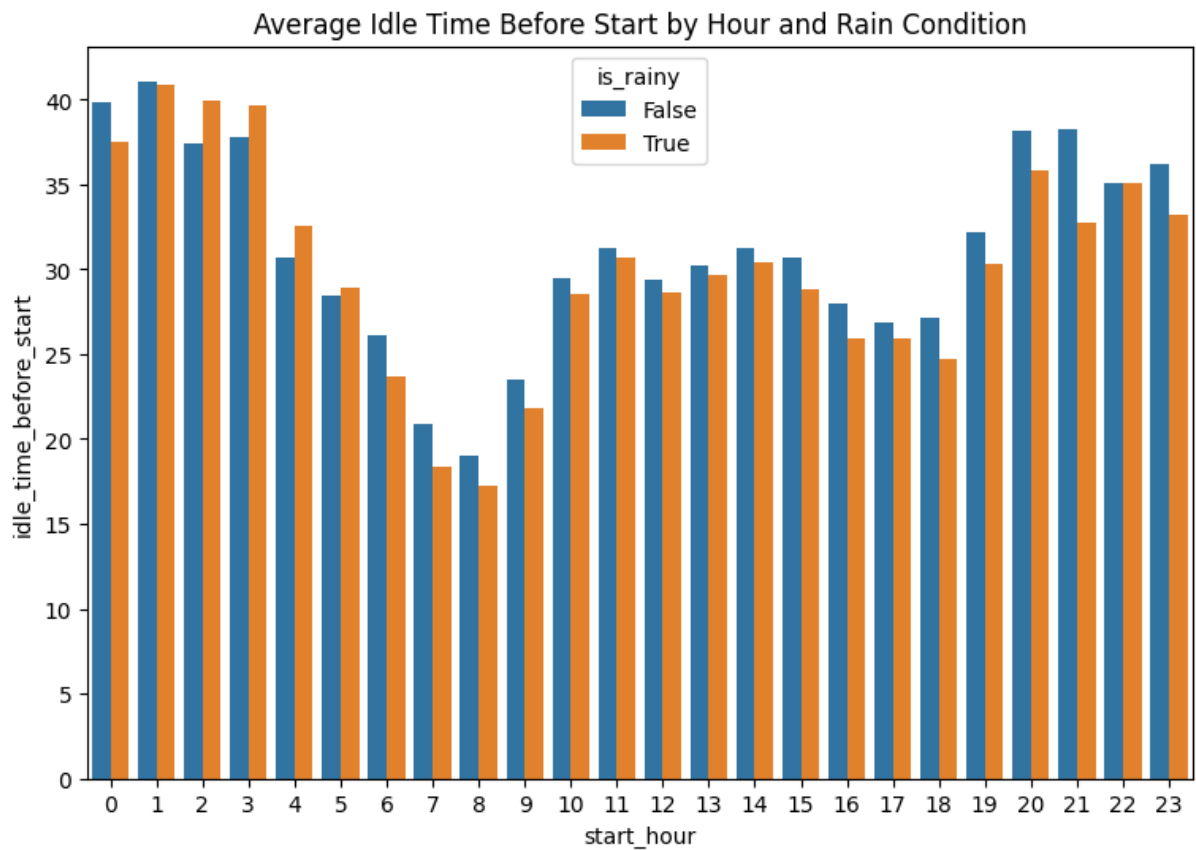


Figure 32: Average idle time by Hour and Rain Condition

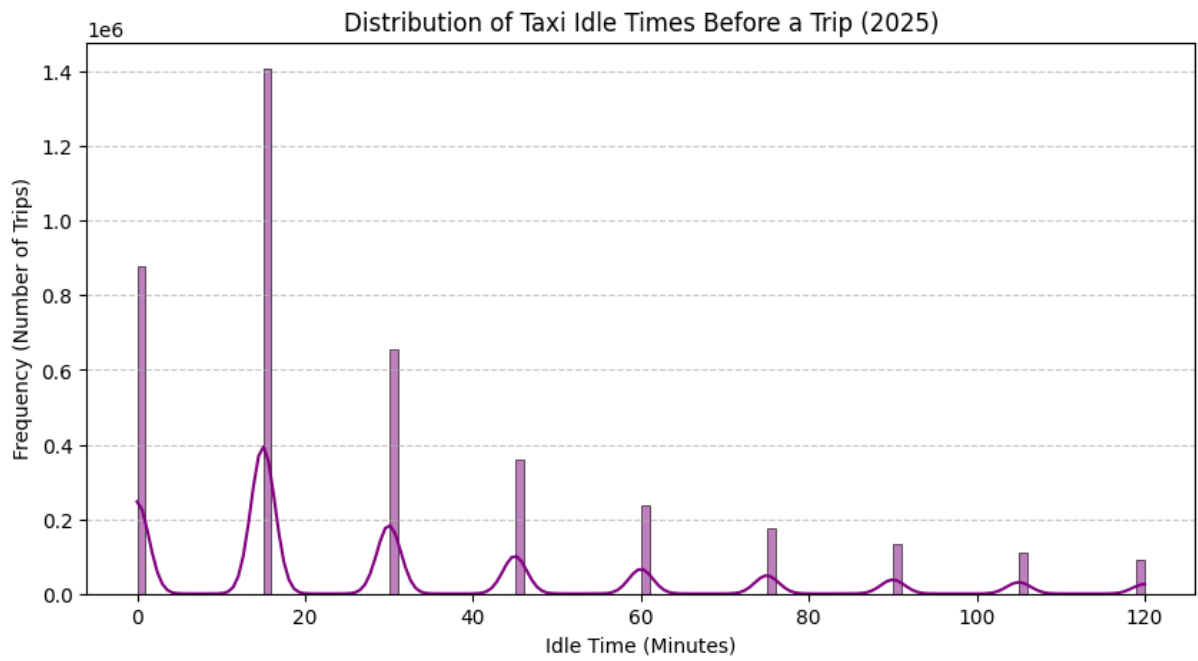


Figure 33: Distribution of Taxi Idle Times before a Trip

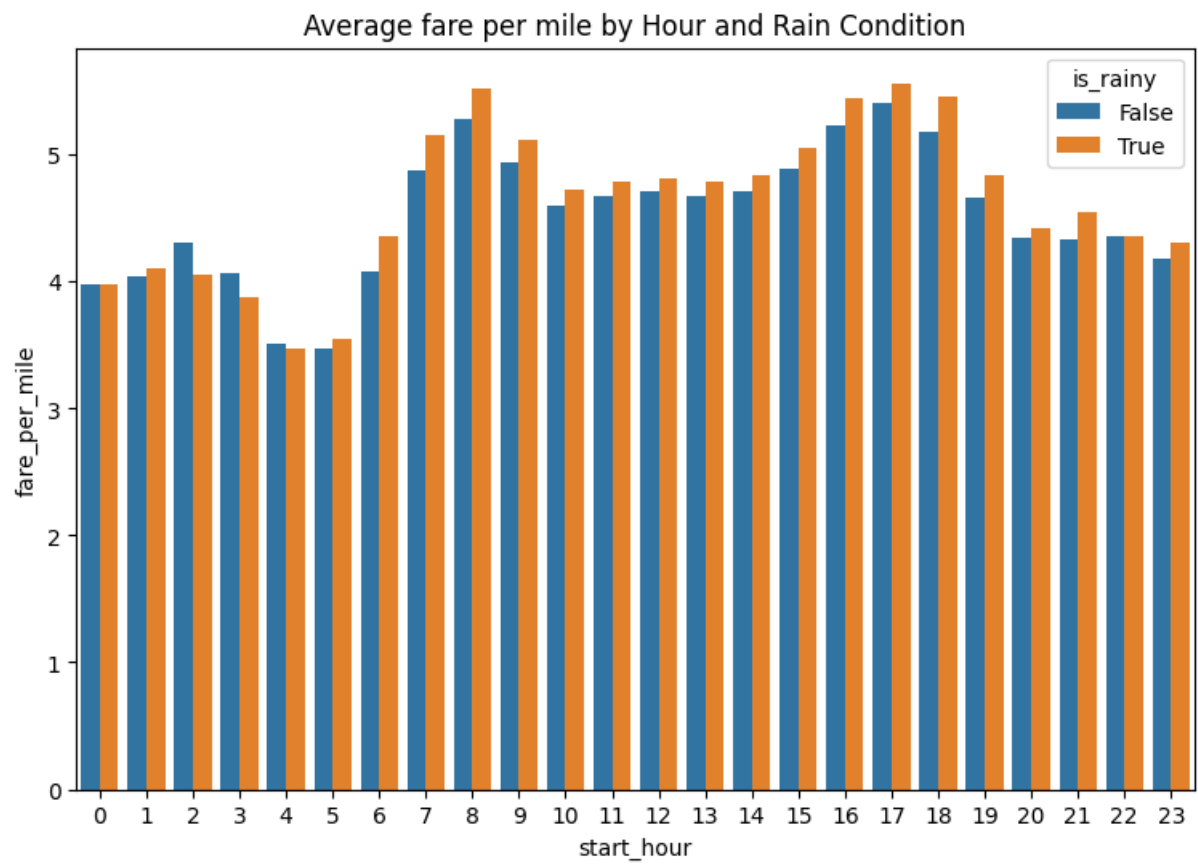


Figure 34: Average fare per mile by Hour and Rain Condition

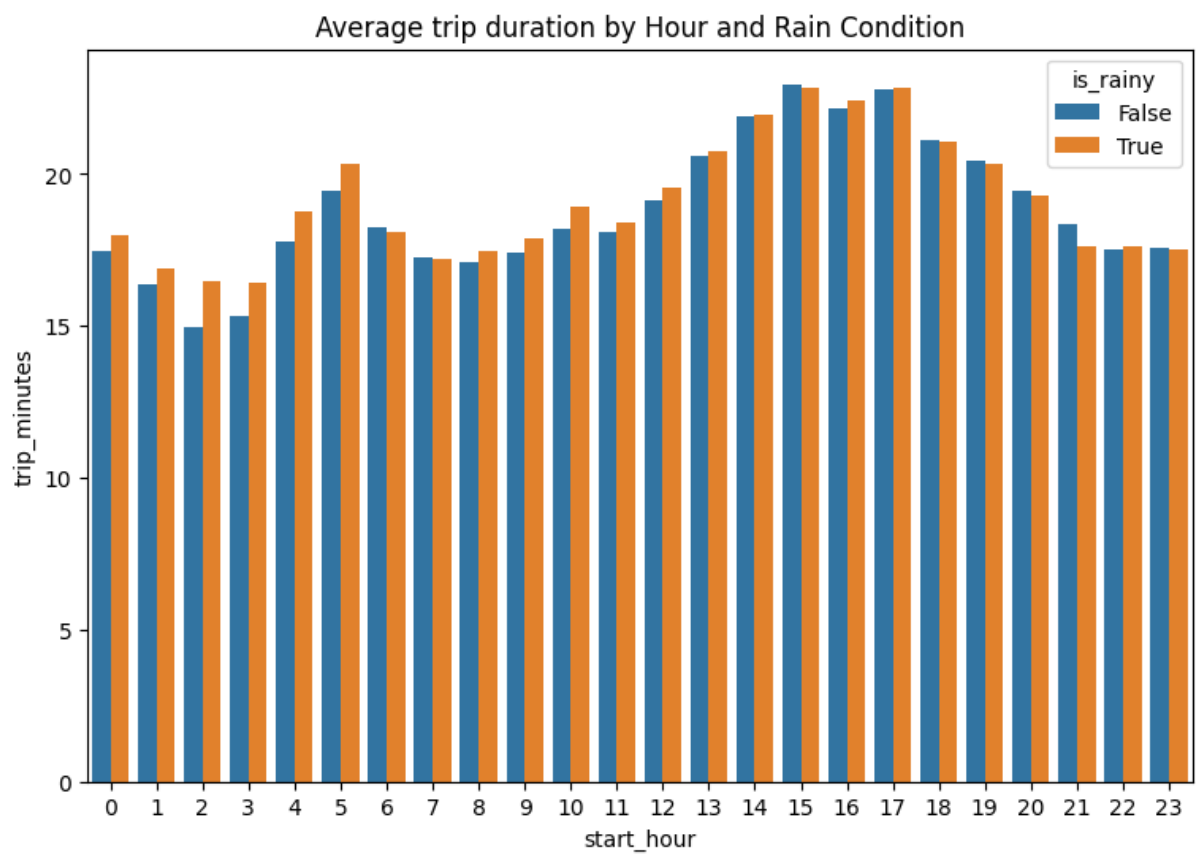


Figure 35: Average trip duration by Hour and Rain Condition

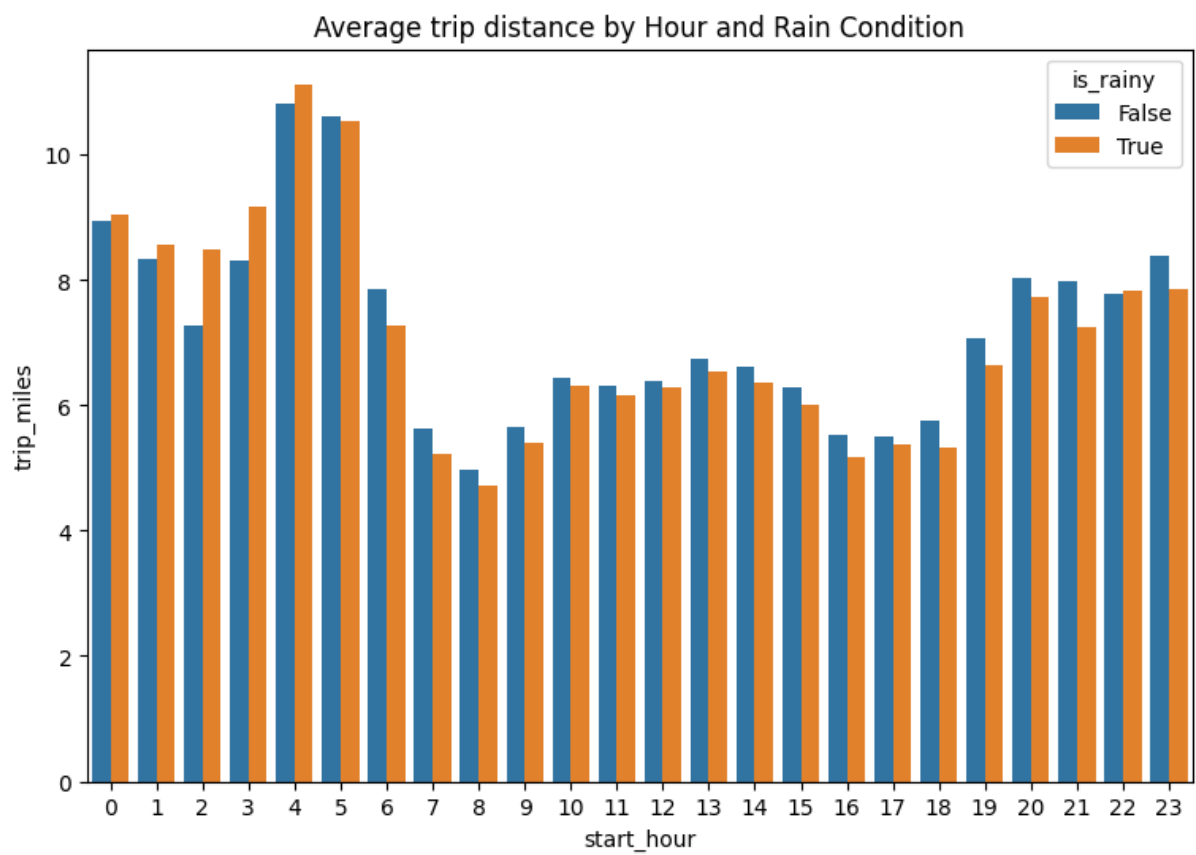


Figure 36: Average trip distance by Hour and Rain Condition

Initiating full comparative Grid Search across 5000 rows...
 Fitting 3 folds for each of 18 candidates, totalling 54 fits
 Master Grid Search completed in 332.34 seconds!

=== THE GRADUAL COMPLEXITY KERNEL LEADERBOARD ===

param_kernel	param_C	param_degree	mean_test_score
rbf	1000	3	0.884359
rbf	1000	2	0.884359
poly	100	3	0.876292
rbf	100	2	0.873112
rbf	100	3	0.873112
poly	10	3	0.865926
poly	1000	2	0.857894
poly	100	2	0.842202
poly	1000	3	0.761338
poly	10	2	0.731756
rbf	10	2	0.586896
rbf	10	3	0.586896
linear	1000	3	0.490885
linear	1000	2	0.490885
linear	100	3	0.490872
linear	100	2	0.490872
linear	10	3	0.490677
linear	10	2	0.490677

Figure 37: Grid Search Results

Structural Configuration	Training Time	Out-of-Sample R^2	MAE (Trips)	Negative Predictions
Linear SVR Baseline (No Kernel)	0.42 seconds	0.6134	13.05	34.52% Outliers
Nystroëm RBF (1.5k, With Weather)	254.67 seconds	0.8316	8.30	28.55% Outliers
Nystroëm RBF (3k, With Weather)	2,164.74 seconds	0.8431	7.51	28.60% Outliers
Nystroëm RBF (1.5k, No Weather)	455.50 seconds	0.8734	6.49	22.28% Outliers

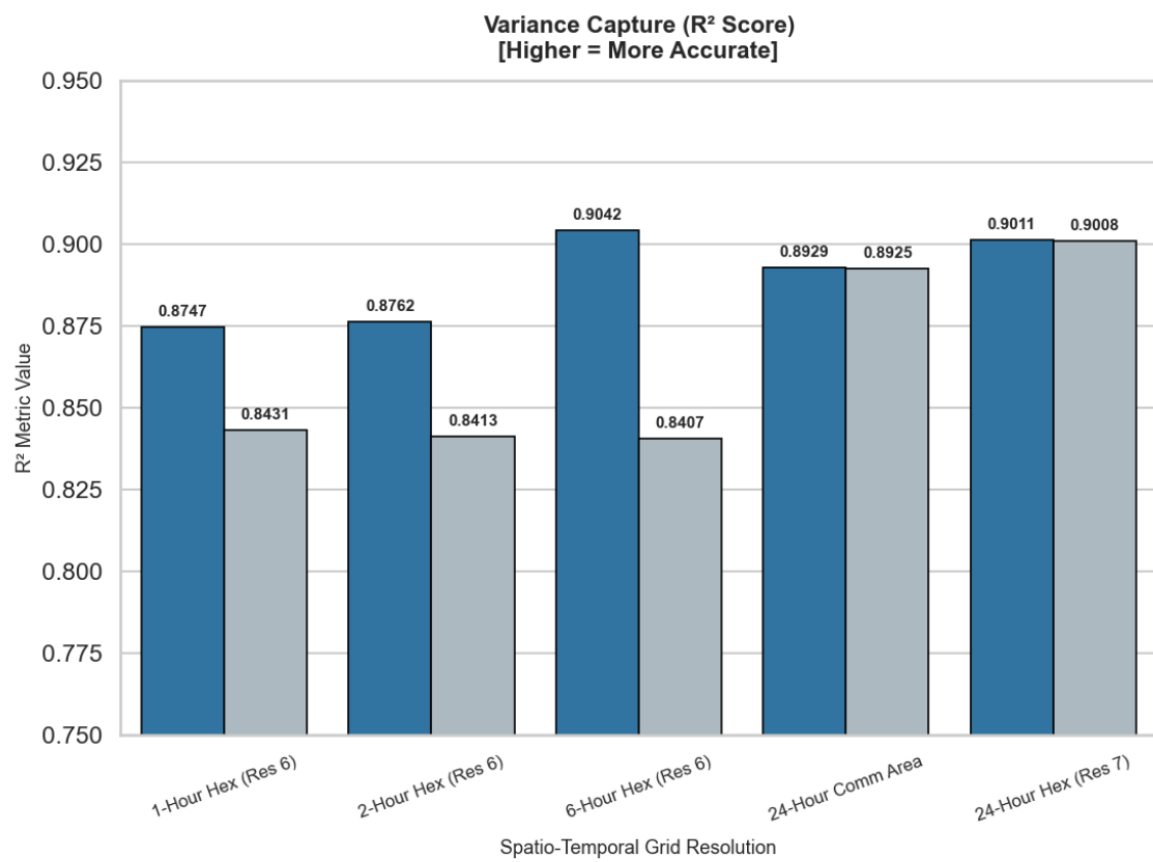
Figure 38: Model Results

```

=====
TWO-STAGE HURDLE SVM PERFORMANCE REPORT
=====
R-Squared ( $R^2$ ):                0.8751
Mean Absolute Error (MAE):         6.44 trips
Root Mean Squared Error (RMSE):    20.31 trips
Normalized RMSE (NRMSE):           112.23%
Mean Actual Test Demand:           18.10 trips
-----
Actual Zero-Demand Rows in Test:   20,723 / 57,816
Hurdle Predicted Zero Rows:        26,400 / 57,816
Negative Predictions Clipped:       0 (Mathematically Eliminated)
=====

```

Figure 39: Hurdle Results

Figure 40: Spatio-temporal model comparison (R^2)

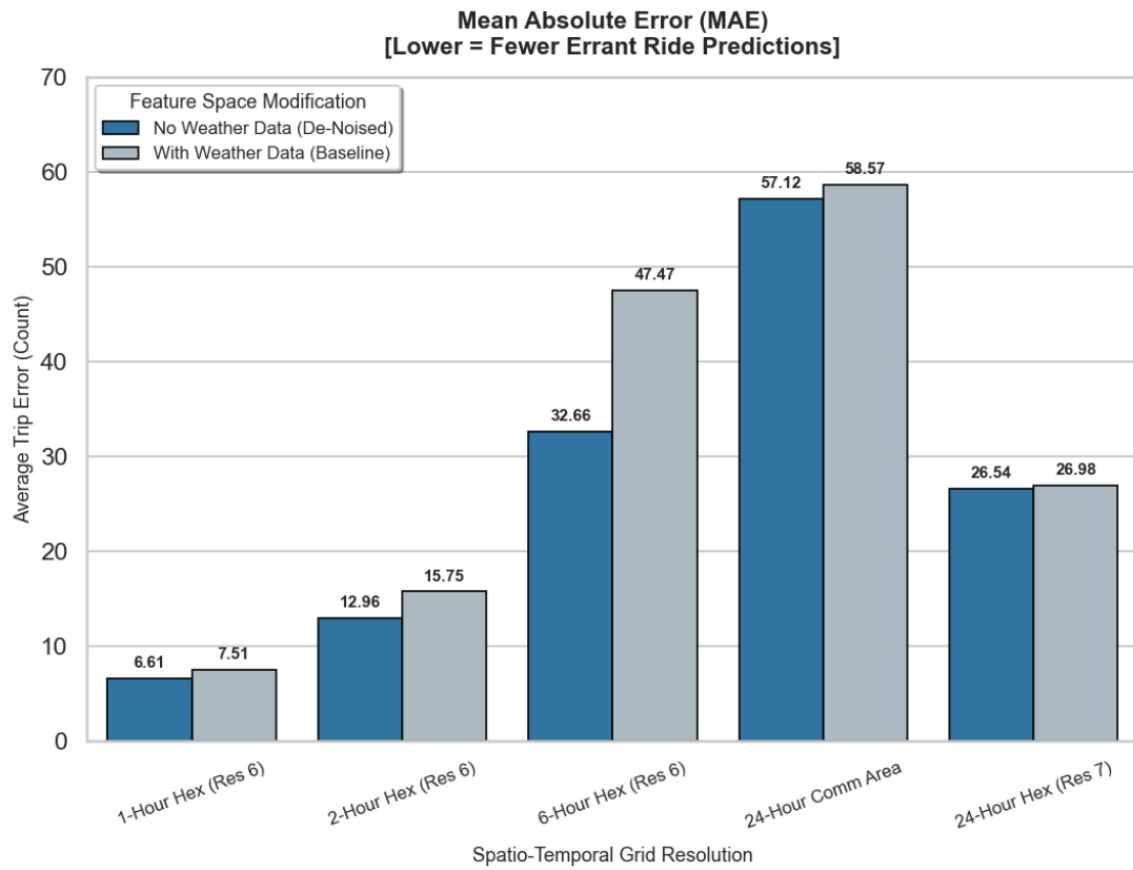


Figure 41: Spatio-temporal model comparison (MAE)

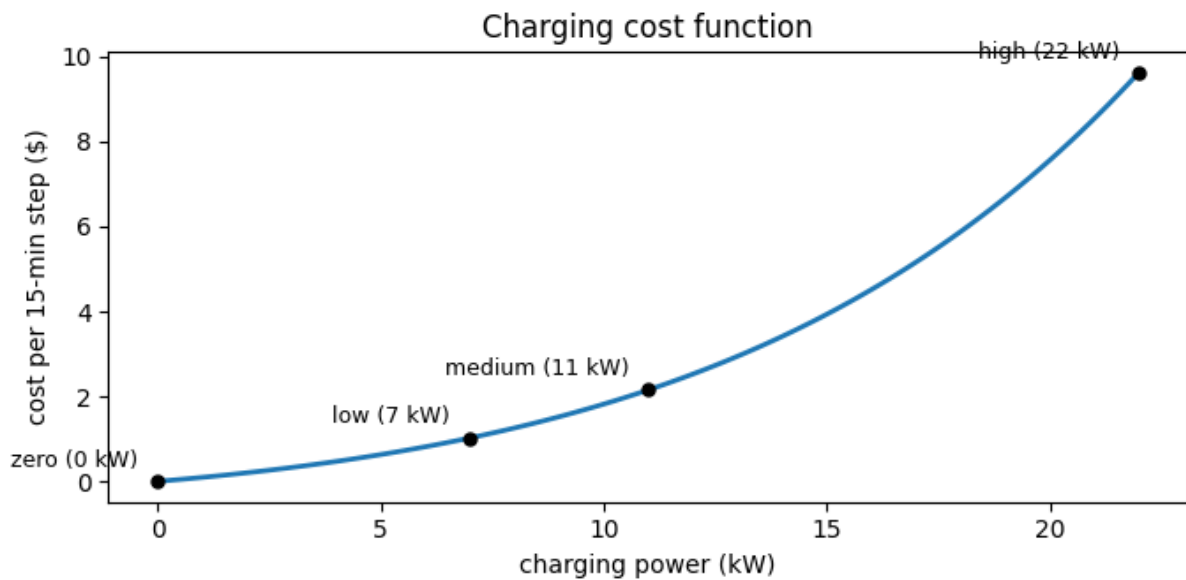


Figure 42: Charging cost as a exponential function of charging power

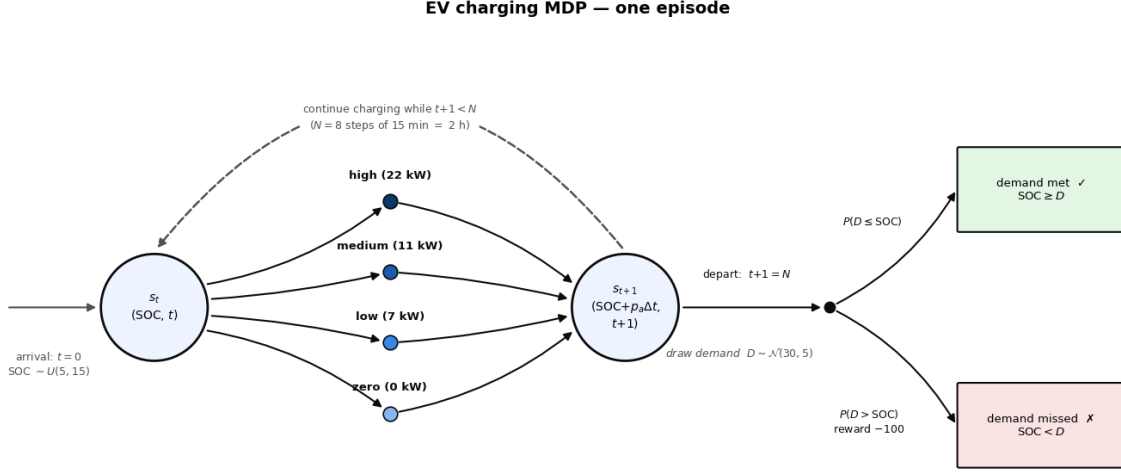


Figure 43: Transition structure of the EV-charging MDP

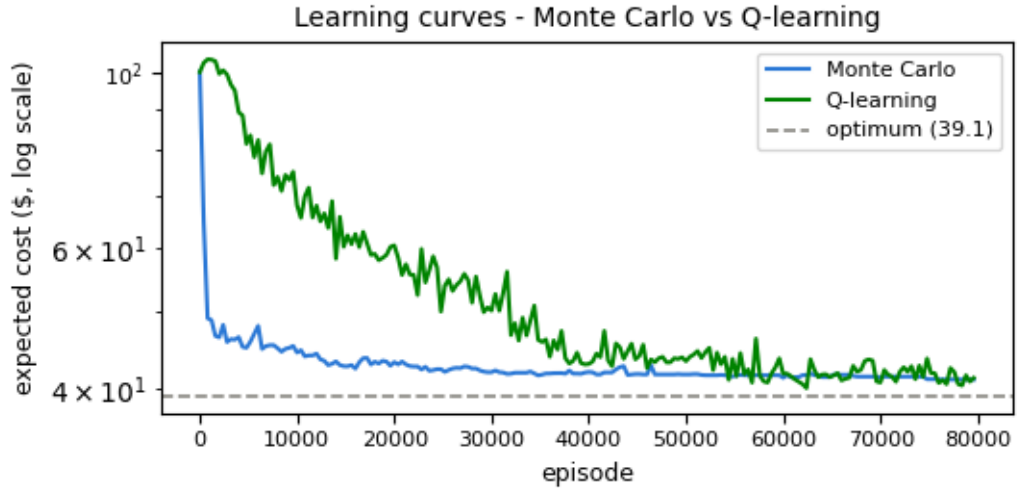


Figure 44: Learning curves of Monte Carlo and Q-learning

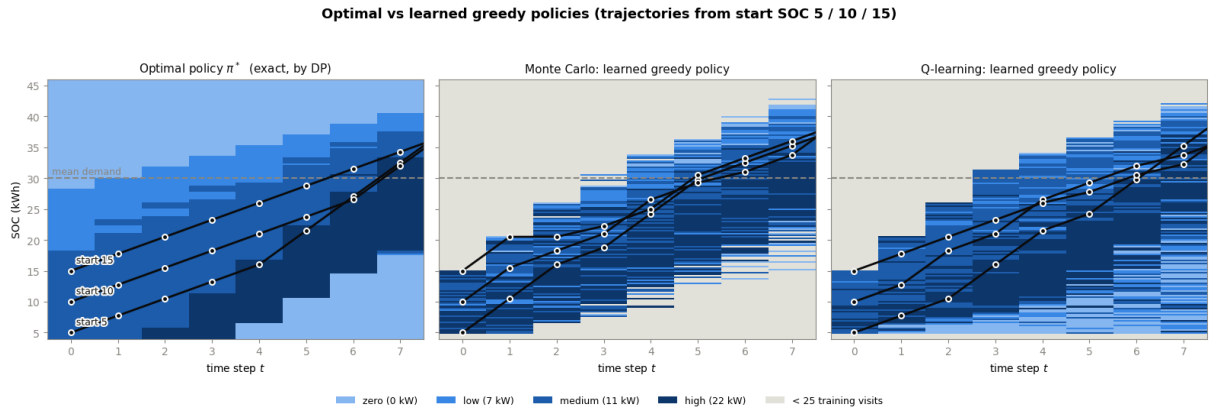


Figure 45: The exact optimal policy next to policies learned by Monte Carlo and Q-learning

6 Appendix B: Use of LLMs

Large language models (LLMs) were used in a supporting role during this project. Claude Code, ChatGpt and Gemini helped with fixing coding errors and answering syntax or programming related questions. Also Claude Code was used to resolved Merge Conflicts and solve git issues and is therefore co-author in the Git Repository with 2 Commits. ChatGPT and Gemini were also used to improve the writing of the report and make passages clearer. Claude Code helped with Quarto questions, including resolving syntax issues and adjusting formatting when needed.